دانشگاه آزاد اسلامی استان قم
دانشکده فنی و مهندسی
گروه مهندسی کامپیوتر

گزارش پروژه کارشناسی مهندسی کامپیوتر

عنوان

طراحی و پیاده‌سازی سامانه متن‌باز مدیریت و تحلیل هزینه بر پایه FinOps با OpenCost و یکپارچه‌سازی با Odoo Community

نگارش
سید امیررضا رضویان

استاد راهنما
لیلا اسماعیلی

مهر ۱۴۰۵

به نام خدا

تعهدنامه اصالت اثر

اینجانب سید امیررضا رضویان متعهد می‌شوم مطالب مندرج در این گزارش پروژه حاصل کار پژوهشی و اجرایی اینجانب تحت نظارت و راهنمایی استاد مربوط بوده و در موارد استفاده از دستاوردها، مستندات و منابع دیگر، ارجاع مناسب در متن و فهرست منابع انجام شده است. این گزارش پیش‌تر برای احراز هیچ مدرک هم‌سطح یا بالاتر ارائه نشده است.

در صورت اثبات تخلف در هر زمان، مسئولیت علمی و آموزشی آن بر عهده اینجانب خواهد بود. استفاده از نتایج علمی و عملی گزارش با ذکر مأخذ و رعایت مقررات دانشگاه بلامانع است.

نام و نام خانوادگی دانشجو: سید امیررضا رضویان

تاریخ: 13 مهر 1405

تقدیر و تشکر

از استاد محترم درس و استاد راهنما که با ارائه بازخوردهای فنی و تاکید بر استفاده از ابزارهای متن‌باز و امکان یکپارچه‌سازی با Odoo مسیر پروژه را مشخص کردند، سپاسگزاری می‌کنم. همچنین از منابع رسمی پروژه‌های OpenCost، Kubernetes، Prometheus، Docker، FastAPI و Odoo که برای یادگیری و پیاده‌سازی مورد استفاده قرار گرفتند تشکر می‌شود.

چکیده

مدیریت هزینه در زیرساخت‌های ابری و کانتینری زمانی ارزش عملی پیدا می‌کند که داده مصرف فنی به شکلی قابل فهم برای تیم‌های مالی و مدیریتی تبدیل شود. در این پروژه یک سامانه متن‌باز با نام FinOps OpenCost Odoo Hub طراحی و پیاده‌سازی شده است که زنجیره‌ای از جمع‌آوری متریک تا تحلیل و ثبت هزینه در سامانه ERP را پوشش می‌دهد. در لایه زیرساخت، یک کلاستر محلی Kubernetes از نوع kind ایجاد و چهار فضای نام نماینده پروژه‌های Commerce-E، Data Platform، CRM و AI Platform راه‌اندازی شد. Prometheus متریک‌های مصرف را جمع‌آوری و OpenCost هزینه تخصیص‌یافته منابع را از طریق Allocation API ارائه می‌کند. در لایه میانی، یک Gateway مبتنی بر FastAPI داده‌های OpenCost را دریافت، نرمال‌سازی و در قالب REST API و داشبورد ارائه می‌کند. این Gateway علاوه بر حالت OpenCost، دارای Mock Provider برای توسعه و آزمون آفلاین است. در لایه سازمانی، افزونه اختصاصی FinOps Connector برای Odoo Community ایجاد شد تا رکوردهای هزینه را دریافت و در نماهای فهرستی، محوری و نموداری تحلیل کند. در فاز ماکت، ۴۸۰ رکورد هزینه بدون ایجاد رکورد تکراری در Odoo همگام‌سازی شد. سپس Docker Compose، WSL2، PostgreSQL، Kubernetes، Helm، Prometheus و OpenCost به صورت عملی راه‌اندازی شدند و حالت Provider از mock به opencost تغییر یافت. در زمان تهیه این گزارش، ارتباط OpenCost با Kubernetes و Gateway تایید شده و یک خطای مرزی در رندر داشبورد هنگام خالی بودن داده‌ها شناسایی و راهکار اصلاح آن مشخص شده است. نتیجه پروژه یک معماری قابل توسعه و مستقل از ابزارهای اختصاصی فروشندگان است که می‌تواند مبنای توسعه برای بودجه‌بندی، توصیه‌های بهینه‌سازی، همگام‌سازی دوره‌ای و استقرار سازمانی قرار گیرد.

واژه‌های کلیدی:

FinOps، OpenCost، Kubernetes، Prometheus، Odoo، FastAPI، Docker

فهرست عناوین

فصل اول: مقدمه و تعریف مسئله 1

۱-۱. مقدمه 2

۱-۲. بیان مسئله 2

۱-۳. ضرورت و اهمیت پروژه 3

۱-۴. اهداف پروژه 4

۱-۵. محدوده و فرضیات 4

۱-۶. روش انجام پروژه 5

۱-۷. ساختار گزارش 5

فصل دوم: مبانی نظری و فناوری‌های مورد استفاده 6

۲-۱. FinOps 7

۲-۲. Kubernetes و کانتینرها 8

۲-۳. Prometheus و OpenCost 9

۲-۴. FastAPI و REST API 10

۲-۵. Odoo Community و PostgreSQL 11

۲-۶. Docker، WSL2 و Helm 12

فصل سوم: تحلیل و طراحی سامانه پیشنهادی 13

۳-۱. نیازمندی‌ها 14

۳-۲. معماری و DFD سطح صفر 15

۳-۳. جریان داده و مدل اطلاعات 16

۳-۴. طراحی API 17

۳-۵. همگام‌سازی و جلوگیری از داده تکراری 18

۳-۶. ملاحظات امنیت و پیکربندی 19

فصل چهارم: پیاده‌سازی و راه‌اندازی 20

۴-۱. راه‌اندازی Gateway 21

۴-۲. Docker و WSL2 23

۴-۳. Odoo و FinOps Connector 25

۴-۴. Kubernetes و Helm 28

۴-۵. Prometheus و OpenCost 30

۴-۶. Workloadهای پروژه 33

۴-۷. تغییر Provider به OpenCost 35

فصل پنجم: آزمون، نتایج و ارزیابی 37

۵-۱. آزمون Gateway 38

۵-۲. آزمون Odoo 39

۵-۳. آزمون Kubernetes و OpenCost 41

۵-۴. نتایج و خطای مرزی Dashboard 43

فصل ششم: جمع‌بندی و کارهای آینده 46

۶-۱. دستاوردها 47

۶-۲. وضعیت تکمیل 48

۶-۳. محدودیت‌ها و توسعه آینده 49

۶-۴. نتیجه‌گیری 50

منابع و مراجع 51

پیوست‌ها 53

فهرست اشکال

شکل ۳-۱. نمودار DFD سطح صفر و جریان کلی داده در سامانه .....

شکل ۴-۱. مستندات خودکار Swagger برای APIهای Gateway .....

شکل ۴-۲. داشبورد ماکت اولیه FinOps .....

شکل ۴-۳. وضعیت Docker و اجرای Compose پس از رفع مشکل WSL2

.....

شکل ۴-۴. شناسایی افزونه FinOps Connector در Odoo Community

.....

شکل ۴-۵. نمایش ۴۸۰ رکورد هزینه در Odoo پس از همگام‌سازی اولیه

.....

شکل ۴-۶. فعال بودن کلاستر Kubernetes از نوع kind در Docker Desktop

.....

شکل ۴-۷. رابط کاربری OpenCost و Cost Summary .....

شکل ۴-۸. Podهای پروژه و سرویس‌های زیرساختی در وضعیت Running

.....

شکل ۴-۹. نمایش namespaceهای پروژه در Cost Allocation .....

شکل ۴-۱۰. تایید mode_provider برابر opencost در Gateway

.....

شکل ۵-۱. خروجی Allocation API برای پروژه‌های Kubernetes

.....

شکل ۵-۲. Traceback خطای ZeroDivisionError در Dashboard

.....

شکل پ-۱ تا پ-۱۱. مجموعه تصاویر تکمیلی مراحل اجرا و عیب‌یابی .....

فهرست جداول

جدول ۲-۱. نقش فناوری‌های اصلی در پروژه .....

جدول ۳-۱. نیازمندی‌های عملکردی سامانه .....

جدول ۳-۲. مدل داده Cost Record .....

جدول ۳-۳. APIهای اصلی Gateway .....

جدول ۳-۴. نگاشت namespace به پروژه سازمانی .....

جدول ۴-۱. منابع درخواستی Workloadهای Kubernetes .....

جدول ۵-۱. نتایج آزمون‌های اصلی .....

جدول ۶-۱. وضعیت تکمیل اجزای پروژه .....

جدول پ-۱. ماتریس خطاها و راه‌حل‌ها .....

فهرست علائم و اختصارات

جدول فهرست علائم-۱. اختصارات پرکاربرد در گزارش

اختصار	عبارت انگلیسی	معادل/کاربرد فارسی
API	Application Programming Interface	رابط برنامه‌نویسی کاربردی
DFD	Data Flow Diagram	نمودار جریان داده
ERP	Enterprise Resource Planning	برنامه‌ریزی منابع سازمان
FinOps	Financial Operations	مدیریت و عملیات مالی فناوری
KPI	Key Performance Indicator	شاخص کلیدی عملکرد
REST	Representational State Transfer	سبک معماری سرویس وب
ORM	Object Relational Mapping	نگاشت شیء به رابطه
CPU	Central Processing Unit	پردازنده
RAM	Random Access Memory	حافظه اصلی
WSL2	Windows Subsystem for Linux 2	زیرسامانه لینوکس ویندوز

1

فصل اول

مقدمه و تعریف مسئله

1 مقدمه و تعریف مسئله

این فصل با معرفی مسئله مدیریت هزینه در زیرساخت‌های ابری و کانتینری آغاز می‌شود و سپس ضرورت پروژه، اهداف، محدوده، روش اجرا و ساختار گزارش را تشریح می‌کند. تمرکز پروژه بر ساخت یک نمونه عملی متن‌باز است که مسیر داده را از زیرساخت فنی تا سامانه سازمانی Odoo پوشش دهد.

1.1 مقدمه

رشد استفاده از رایانش ابری و معماری‌های مبتنی بر کانتینر باعث شده هزینه فناوری اطلاعات به شکل مستقیم به الگوی مصرف منابع وابسته شود. در چنین شرایطی صرف مشاهده مبلغ کل هزینه کافی نیست و سازمان نیاز دارد هزینه را به پروژه، تیم، سرویس و محیط اجرایی نسبت دهد. رویکرد FinOps برای ایجاد زبان مشترک میان تیم‌های فنی، مالی و مدیریتی شکل گرفته است تا مصرف و ارزش حاصل از زیرساخت به صورت قابل اندازه‌گیری مدیریت شود [1].

در محیط‌های Kubernetes، منابع پردازشی به صورت پویا میان Podها و namespaceها توزیع می‌شوند. بنابراین ابزار مدیریت هزینه باید بتواند متریک‌های مصرف CPU، حافظه، ذخیره‌سازی و شبکه را به هزینه قابل تخصیص تبدیل کند. OpenCost یکی از ابزارهای متن‌باز تخصصی برای این منظور است [2].

1.2 بیان مسئله

مسئله اصلی پروژه، نبود یک مسیر یکپارچه و متن‌باز برای جمع‌آوری داده مصرف Kubernetes، محاسبه هزینه، ارائه گزارش تحلیلی و انتقال نتیجه به یک سامانه ERP است. بسیاری از ابزارهای تجاری مدیریت هزینه به ارائه‌دهنده خاص یا سرویس ابری مشخص وابسته هستند و در یک پروژه دانشگاهی که معیار استفاده از ابزار متن‌باز است، انتخاب مناسبی محسوب نمی‌شوند.

از سوی دیگر، داده خام OpenCost به تنهایی برای مصرف مدیریتی کافی نیست. خروجی باید نرمال‌سازی شود، قابلیت گزارش‌گیری و API داشته باشد، از داده آزمایشی برای توسعه پشتیبانی کند و

در نهایت به Odoo منتقل شود. به همین دلیل یک Gateway مستقل میان OpenCost و Odoo طراحی شد تا وابستگی لایه‌ها کاهش یابد.

1.3 ضرورت و اهمیت پروژه

اهمیت این پروژه از سه جنبه قابل بررسی است. نخست، از منظر فنی، Kubernetes و معماری کانتینری نیازمند ابزار مشاهده‌پذیری و تحلیل هزینه متناسب با ماهیت پویای منابع هستند. دوم، از منظر مدیریتی، هزینه باید به پروژه و واحد سازمانی نسبت داده شود تا تصمیم‌گیری بودجه‌ای و بهینه‌سازی امکان‌پذیر باشد. سوم، از منظر مهندسی نرم‌افزار، اتصال یک ابزار تخصصی مانند OpenCost به ERP متن‌باز Odoo نمونه‌ای از یکپارچه‌سازی واقعی بین سامانه زیرساختی و سامانه سازمانی است.

1.4 اهداف پروژه

• طراحی معماری متن‌باز برای مدیریت و تخصیص هزینه بر پایه FinOps.

• راه‌اندازی Kubernetes، Prometheus و OpenCost در محیط محلی و دریافت Cost Allocation واقعی.

• توسعه Gateway مبتنی بر FastAPI برای دریافت، نرمال‌سازی، ذخیره و ارائه داده هزینه.

• پشتیبانی هم‌زمان از Mock Provider برای توسعه و OpenCost Provider برای داده واقعی.

• ایجاد افزونه FinOps Connector برای Odoo Community و نمایش داده در نماهای List، Pivot و Graph.

• پیاده‌سازی شناسه پایدار و منطق Upsert برای جلوگیری از رکوردهای تکراری.

• ثبت فرایند نصب، عیب‌یابی و نتیجه به گونه‌ای که پروژه قابل تکرار و ارائه باشد.

1.5 محدوده، فرضیات و محدودیت‌ها

نسخه فعلی پروژه در Windows و Docker Desktop اجرا شده و کلاستر Kubernetes از نوع kind و تک‌گرهی است. بنابراین اعداد هزینه‌ای OpenCost در این محیط، صورتحساب واقعی AWS، Azure یا GCP نیستند، بلکه تخصیص هزینه بر اساس مدل قیمت‌گذاری و درخواست/مصرف منابع در کلاستر محلی هستند. هدف فاز حاضر اثبات زنجیره فنی End-to-End و قابلیت یکپارچه‌سازی است.

در Odoo، داده‌های فاز ماکت با Provider برابر Mock-OpenCost نگهداری شده‌اند و در فاز نهایی باید پیش از ورود داده واقعی پاک‌سازی شوند. همچنین Budget باید به عنوان داده مدیریتی مستقل از Cost Provider نگهداری شود؛ زیرا OpenCost وظیفه محاسبه Cost را دارد و منبع بودجه سازمانی نیست.

1.6 روش انجام پروژه

پروژه به صورت افزایشی انجام شد. ابتدا Gateway و Dashboard با داده ماکت پیاده‌سازی شدند تا مدل داده و API تثبیت شود. سپس Docker Compose و Odoo Community راه‌اندازی و افزونه اختصاصی نصب شد. پس از تایید Sync و Idempotency، کلاستر Kubernetes، Prometheus و OpenCost اضافه شدند. در مرحله بعد namespaceهای پروژه و Resource Requestهای متفاوت ایجاد شد تا OpenCost داده واقعی تخصیص هزینه تولید کند. در نهایت Provider فعال Gateway از mock به opencost تغییر یافت و وضعیت سرویس از طریق endpoint سلامت بررسی شد.

1.7 ساختار گزارش

فصل دوم مبانی نظری و ابزارهای مورد استفاده را معرفی می‌کند. فصل سوم نیازمندی‌ها، معماری، نمودار جریان داده و مدل اطلاعات را توضیح می‌دهد. فصل چهارم مراحل پیاده‌سازی و راه‌اندازی را با تصاویر واقعی پروژه مستند می‌کند. فصل پنجم آزمون‌ها و نتایج را ارائه می‌دهد و فصل ششم جمع‌بندی، وضعیت تکمیل و مسیر توسعه آینده را بیان می‌کند. دستورات کامل و مستند تصویری تکمیلی نیز در پیوست‌ها قرار گرفته‌اند.

2

فصل دوم

مبانی نظری و فناوری‌های مورد استفاده

2 مبانی نظری و فناوری‌های مورد استفاده

در این فصل مفاهیم و ابزارهایی معرفی می‌شوند که معماری پروژه بر پایه آن‌ها شکل گرفته است. هدف، فراهم‌کردن زمینه فنی لازم برای درک تصمیم‌های طراحی و مراحل پیاده‌سازی فصل‌های بعد است.

2.1 FinOps

FinOps یک چارچوب عملیاتی و فرهنگی برای بیشینه‌کردن ارزش حاصل از هزینه فناوری و Cloud است. در این رویکرد، تیم‌های فنی، مالی و کسب‌وکار بر مبنای داده مشترک درباره مصرف، تخصیص، بودجه و بهینه‌سازی تصمیم می‌گیرند [1]. سه حوزه کلی Inform، Optimize و Operate را می‌توان در پروژه حاضر به ترتیب با مشاهده و تخصیص هزینه، شناسایی فرصت بهینه‌سازی و اجرای همگام‌سازی/عملیات دوره‌ای مرتبط دانست.

2.2 Kubernetes و کانتینرها

Kubernetes سامانه ارکستراسیون کانتینرهاست و منابعی مانند Node، Pod، Deployment، Service و Namespace را مدیریت می‌کند [3]. Namespace در این پروژه نقش مرز منطقی پروژه سازمانی را دارد؛ به گونه‌ای که commerce-e، platform-data، crm و platform-ai هرکدام به یک پروژه در Gateway و Odoo نگاشت می‌شوند.

2.3 Prometheus و OpenCost

Prometheus یک سامانه متن‌باز برای جمع‌آوری و پرس‌وجوی متریک است [4]. OpenCost متریک‌های Kubernetes را از Prometheus دریافت و آن‌ها را به Cost Allocation تبدیل می‌کند [2]. API تخصیص هزینه OpenCost امکان تجمیع داده بر اساس namespace، Pod، Container و سایر ابعاد را فراهم می‌کند. در این پروژه aggregation بر اساس namespace انتخاب شده تا نگاشت مستقیم به پروژه‌های سازمانی امکان‌پذیر باشد.

2.4 FastAPI و REST API

Gateway پروژه با Python و FastAPI پیاده‌سازی شده است. FastAPI علاوه بر ساخت REST API، مستندات تعاملی OpenAPI/Swagger را به صورت خودکار تولید می‌کند [6]. این ویژگی در فاز توسعه برای تست مستقل endpointها و مشاهده schemaهای ورودی و خروجی اهمیت زیادی داشت.

2.5 Odoo Community و PostgreSQL

Odoo Community یک ERP متن‌باز و توسعه‌پذیر است [5]. افزونه اختصاصی FinOps Connector در این پروژه مدل Cost Record، منوی FinOps، عملیات Sync و نماهای تحلیلی را به Odoo اضافه می‌کند. Odoo داده‌های خود را در PostgreSQL نگه می‌دارد؛ در حالی که Gateway برای نمونه دانشگاهی از SQLite استفاده می‌کند تا استقرار آن سبک و مستقل باشد.

2.6 Docker، WSL۲ و Helm

Docker Compose برای اجرای هم‌زمان PostgreSQL، Odoo و Gateway استفاده شده است [8]. روی Windows، Docker Desktop از WSL2 به عنوان backend لینوکسی استفاده می‌کند. در لایه Kubernetes، Helm به عنوان مدیر بسته برای نصب Prometheus و OpenCost استفاده شده است [9].

جدول ۲-۱. نقش فناوری‌های اصلی در پروژه

فناوری	نقش در پروژه	وضعیت
FinOps	چارچوب مدیریت، تخصیص و بهینه‌سازی هزینه	مبنای مفهومی
OpenCost	محاسبه Cost Allocation در Kubernetes	راه‌اندازی شده
Prometheus	جمع‌آوری متریک Kubernetes	راه‌اندازی شده

Kubernetes kind	اجرای Workloadهای محلی	راه‌اندازی شده
FastAPI	Gateway و REST API	پیاده‌سازی شده
SQLite	ذخیره داده Gateway	پیاده‌سازی شده
Odoo Community	ERP و تحلیل سازمانی هزینه	راه‌اندازی شده
PostgreSQL	پایگاه داده Odoo	راه‌اندازی شده
Docker Compose	اجرای چندسرویسی	راه‌اندازی شده
Helm	نصب سرویس‌های Kubernetes	راه‌اندازی شده

3

فصل سوم

تحلیل و طراحی سامانه پیشنهادی

3 تحلیل و طراحی سامانه پیشنهادی

معماری پروژه بر اصل جداسازی مسئولیت‌ها استوار است. OpenCost منبع هزینه و مصرف زیرساخت است، Gateway مسئول نرمال‌سازی و ارائه API است، و Odoo لایه سازمانی برای ذخیره و تحلیل هزینه در کنار موجودیت‌های کسب‌وکار محسوب می‌شود. این جداسازی امکان جایگزینی Provider یا توسعه مصرف‌کنندگان دیگر API را فراهم می‌کند.

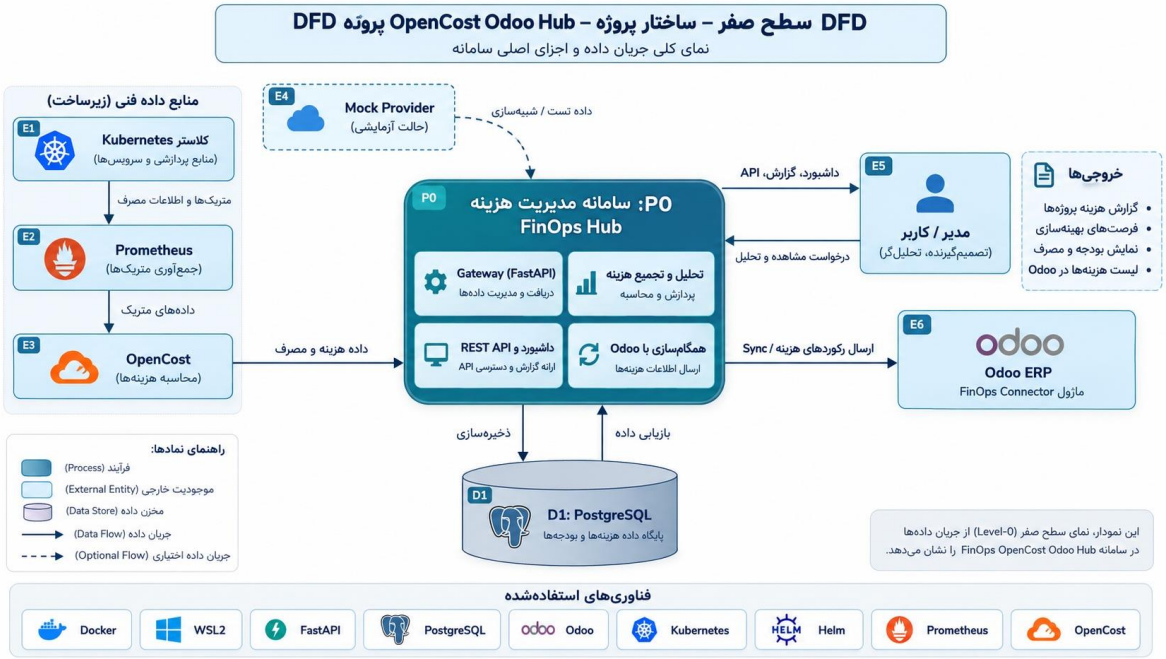
3.1 نیازمندی‌های عملکردی

جدول ۳-۱. نیازمندی‌های عملکردی سامانه

شناسه	نیازمندی
01-FR	دریافت Cost Allocation از OpenCost
02-FR	پشتیبانی از Mock و OpenCost Provider
03-FR	نرمال‌سازی پروژه، سرویس، هزینه، تاریخ و منبع
04-FR	ارائه API خلاصه، هزینه، بودجه و Recommendation
05-FR	ارائه خروجی مناسب برای Odoo
06-FR	ذخیره Cost Record و جلوگیری از Duplicate
07-FR	نمایش List، Pivot و Graph در Odoo
08-FR	امکان Sync دستی و توسعه برای Sync دوره‌ای
09-FR	Dashboard مستقل Gateway
10-FR	قابلیت توسعه Budget و Recommendation مستقل از داده ماکت

3.2 معماری و DFD سطح صفر

نمودار سطح صفر جریان داده، اجزای اصلی و ارتباط میان آن‌ها را نشان می‌دهد. متریک‌های Kubernetes توسط Prometheus جمع‌آوری و در OpenCost به داده هزینه تبدیل می‌شوند. Gateway این داده را دریافت می‌کند، در مخزن داخلی نگه می‌دارد و برای Dashboard و Odoo ارائه می‌کند. Mock Provider نیز به عنوان جریان اختیاری برای توسعه و آزمون وجود دارد.



شکل ۳-۱. نمودار DFD سطح صفر و جریان کلی داده در سامانه

3.3 جریان داده و مدل اطلاعات

در حالت واقعی، OpenCost API با پارامتر namespace=aggregate فراخوانی می‌شود. هر namespace از طریق جدول نگاشت به نام پروژه و Department سازمانی تبدیل می‌شود. سپس هزینه‌های CPU، RAM، ذخیره‌سازی و شبکه به رکوردهای استاندارد Cost Record تبدیل و با id_external پایدار ذخیره می‌شوند.

جدول ۳-۲. مدل داده Cost Record

فیلد	کاربرد
id_external	شناسه پایدار برای Upsert و جلوگیری از Duplicate
date_usage	تاریخ مصرف
provider	OpenCost یا Mock–OpenCost
id_account	شناسه Cluster یا Account
service	CPU، RAM، Storage، Network و ...
category	گروه هزینه
resource	Namespace یا Resource
project	نام پروژه کسب‌وکار
department	واحد سازمانی
environment	محیط اجرا
currency	واحد پول
cost	مقدار هزینه
source	منبع داده و روش محاسبه

3.4 طراحی API

جدول ۳-۳. APIهای اصلی Gateway

Method	مسیر	کاربرد
GET	health/	سلامت سرویس و Provider فعال
GET	summary/v1/api/	خلاصه هزینه و KPIها
GET	costs/v1/api/	رکوردهای هزینه با فیلتر
GET	budgets/v1/api/	بودجه پروژه‌ها
GET	recommendations/v1/api/	پیشنهادهای بهینه‌سازی
POST	provider/sync/v1/api/	همگام‌سازی منبع داده
GET	export/odoo/v1/api/	خروجی نرمال‌شده برای Odoo
GET	/	Dashboard

3.5 همگام‌سازی و جلوگیری از داده تکراری

برای هر رکورد یک id_external پایدار تولید می‌شود. در عملیات Sync اگر رکوردی با همان شناسه وجود داشته باشد، داده موجود Update می‌شود و در غیر این صورت Create انجام می‌گیرد. این الگو Upsert نام دارد و باعث می‌شود اجرای چندباره Sync باعث افزایش مصنوعی تعداد رکوردها نشود. همین رفتار در Odoo نیز با ثابت ماندن تعداد ۴۸۰ رکورد در Sync مجدد تایید شد.

جدول ۳-۴. نگاشت namespace به پروژه سازمانی

Namespace	Project در Odoo/Gateway	Department
commerce-e	Commerce-E	Digital
platform-data	Data Platform	Data
crm	CRM	Sales
platform-ai	AI Platform	AI

3.6 ملاحظات امنیت و پیکربندی

تنظیمات محیطی در فایل .env ریشه Docker Compose نگهداری می‌شوند و نباید رمزهای حساس در سورس کد ثبت شوند. Odoo دارای Master Password جدا از حساب مدیر است؛ بنابراین تصویر مرحله ساخت دیتابیس در مستند حاضر با حذف مقدار رمز نمایش داده شده است. در نسخه سازمانی، Authentication برای API، مدیریت Secrets و محدودسازی دسترسی شبکه باید اضافه شود.

4

فصل چهارم

پیاده‌سازی و راه‌اندازی

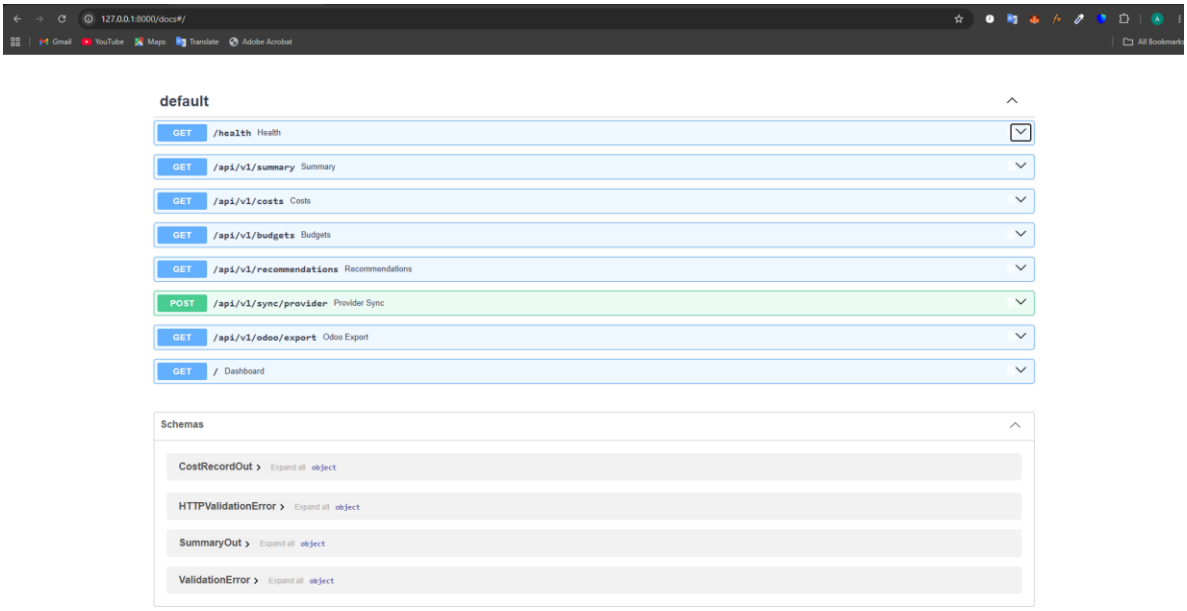
4 پیاده‌سازی و راه‌اندازی

در این فصل روند عملی اجرای پروژه از Gateway محلی تا OpenCost واقعی تشریح می‌شود. تصاویر این فصل مستقیماً از اجرای پروژه تهیه شده‌اند و برای هر مرحله، نتیجه و خطاهای مهم نیز توضیح داده می‌شود.

4.1 راه‌اندازی Gateway و حالت ماکت

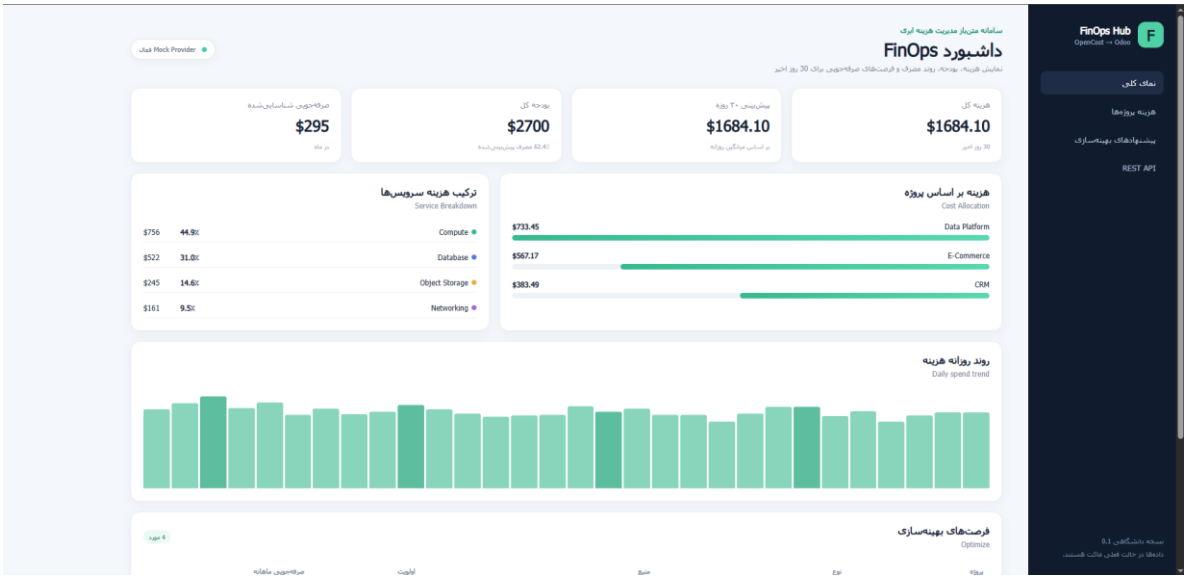
ابتدا محیط مجازی Python در پوشه gateway ایجاد و وابستگی‌ها نصب شد. سپس Uvicorn برنامه FastAPI را روی پورت 8000 اجرا کرد. مستندات Swagger در مسیر /docs مجموعه endpointهای سلامت، Summary، Costs، Budgets، Recommendations، Sync Provider و Odoo Export را نمایش داد.

```
cd gateway
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload
```



شکل ۴-۱. مستندات خودکار Swagger برای APIهای Gateway

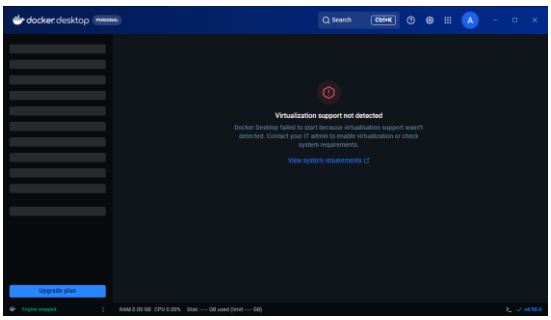
در حالت Mock، داده‌های هزینه برای چهار پروژه و چند سرویس تولید شد و Dashboard مستقل Gateway وضعیت هزینه کل، بودجه، پیش‌بینی، توزیع هزینه پروژه‌ها و فرصت‌های بهینه‌سازی را نمایش داد. این فاز پیش از ورود OpenCost، برای تثبیت مدل داده و UI ضروری بود.

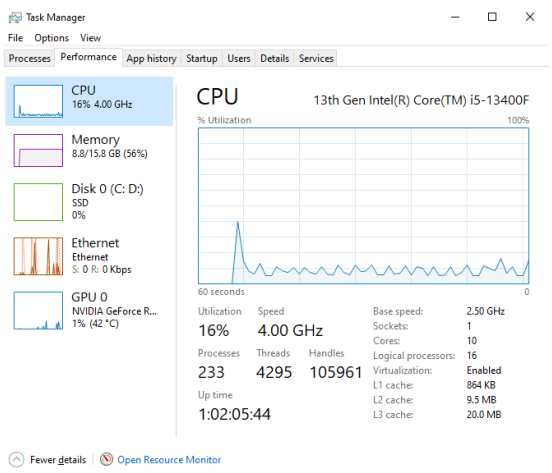


شکل ۴-۲. داشبورد ماکت اولیه FinOps

4.2 Docker Desktop، WSL۲ و Compose

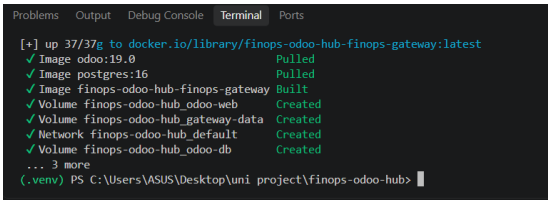
در اولین اجرای Docker Desktop، برنامه خطای Virtualization support not detected نشان داد. بررسی Task Manager مشخص کرد Virtualization در BIOS فعال است؛ بنابراین مسیر عیب‌یابی به WSL2 و Kernel آن محدود شد. پس از فعال‌سازی Windows Subsystem for Linux، VirtualMachinePlatform و به‌روزرسانی Kernel با download-web، وضعیت WSL2 به نسخه 2 تغییر کرد و Docker Engine اجرا شد.

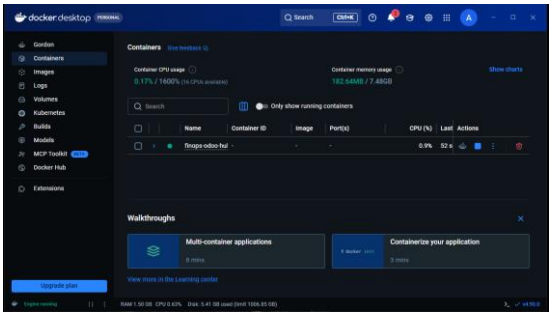




شکل ۴-۳. عیب‌یابی Docker: خطای Virtualization و تایید فعال بودن قابلیت مجازی‌سازی

پس از آماده‌شدن Docker، فایل yml.compose-docker برای اجرای سه مولفه PostgreSQL، Odoo و FinOps Gateway استفاده شد. Imageهای Odoo و PostgreSQL دریافت، Gateway Build و Network/Volumeهای لازم ایجاد شدند.

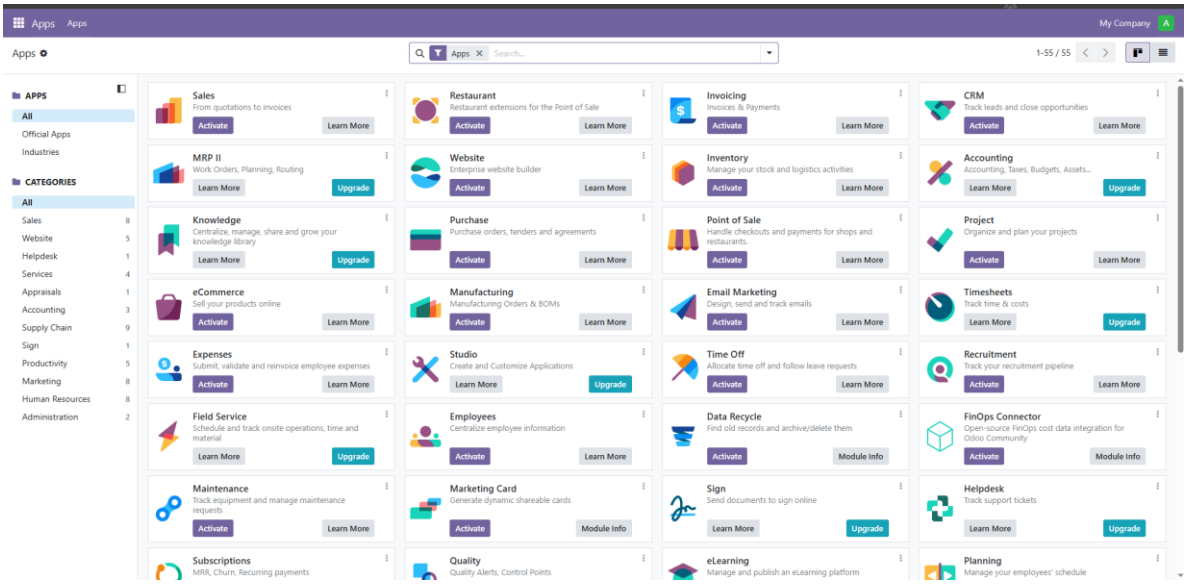




شکل ۴-۴. ساخت سرویس‌ها با Docker Compose و فعال شدن Docker Engine

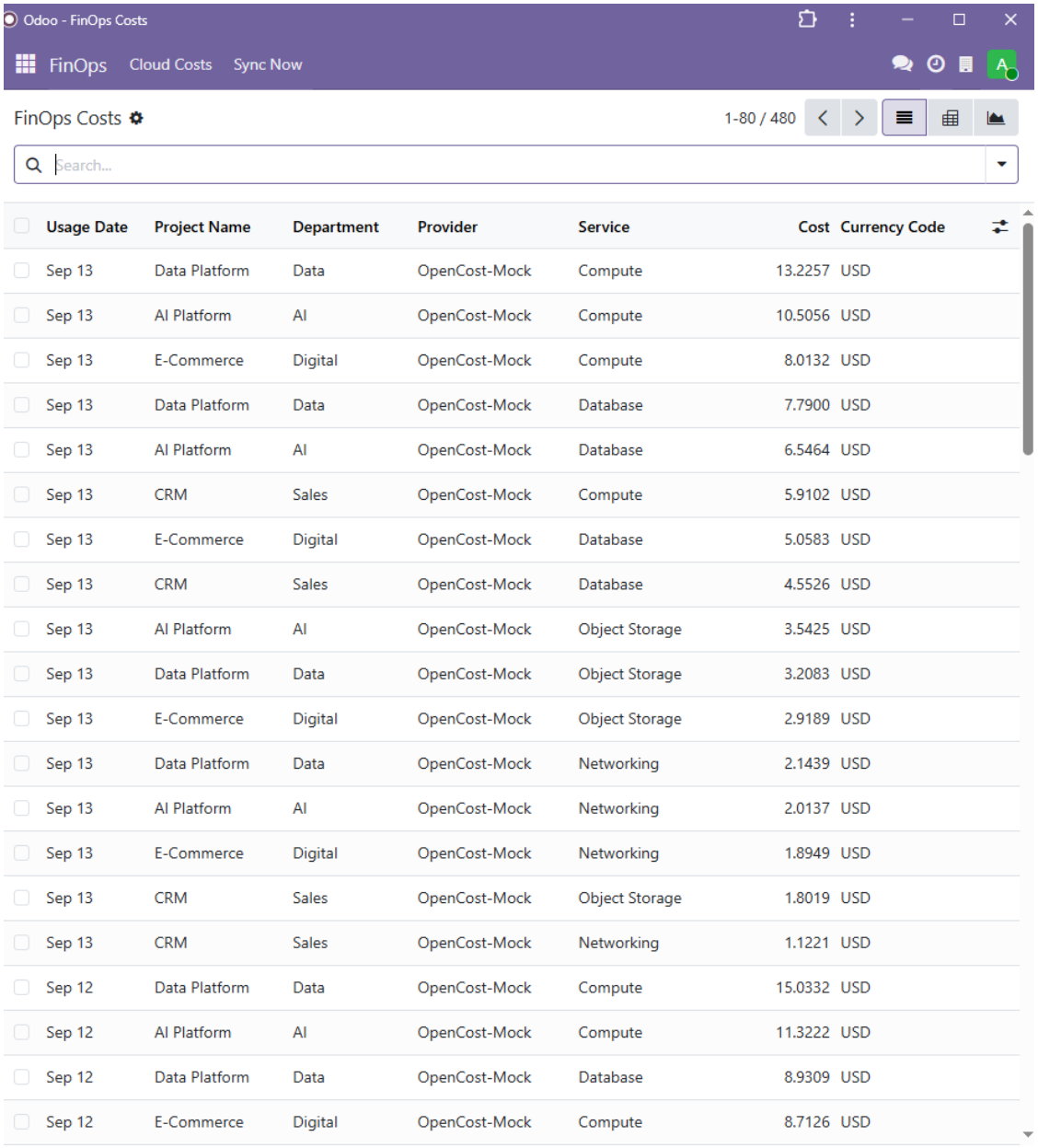
4.3 Odoo Community و FinOps Connector

Odoo 19 Community روی پورت 8069 در دسترس قرار گرفت. دیتابیس پروژه بدون Demo Data ایجاد شد و پوشه addons_odoo به مسیر افزونه‌های Odoo Mount شد. بنابراین افزونه FinOps Connector در فهرست Apps شناسایی و فعال گردید.



شکل ۴-۵. شناسایی افزونه FinOps Connector در Odoo Community

پس از نصب افزونه، منوی FinOps و صفحه Cloud Costs در Odoo ایجاد شد. با داده ماکت، ترکیب ۳۰ روز، چهار پروژه و چهار سرویس برابر با ۴۸۰ رکورد بود. اجرای مجدد Sync تعداد رکوردها را افزایش نداد و منطق Idempotent تایید شد.

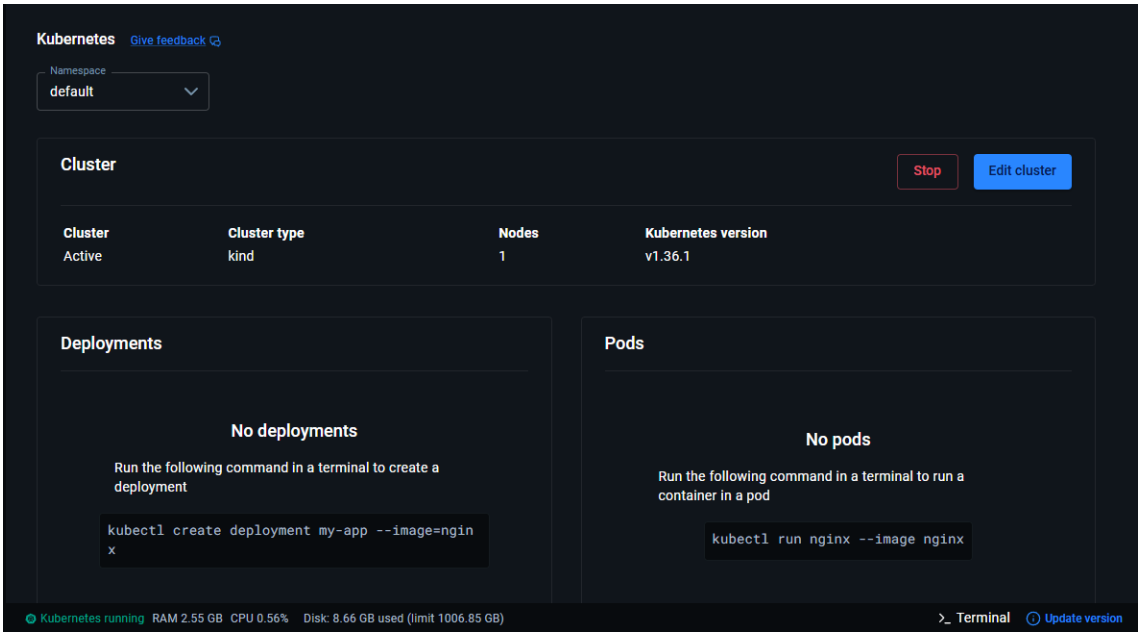


Usage Date	Project Name	Department	Provider	Service	Cost	Currency Code
Sep 13	Data Platform	Data	OpenCost-Mock	Compute	13.2257	USD
Sep 13	AI Platform	AI	OpenCost-Mock	Compute	10.5056	USD
Sep 13	E-Commerce	Digital	OpenCost-Mock	Compute	8.0132	USD
Sep 13	Data Platform	Data	OpenCost-Mock	Database	7.7900	USD
Sep 13	AI Platform	AI	OpenCost-Mock	Database	6.5464	USD
Sep 13	CRM	Sales	OpenCost-Mock	Compute	5.9102	USD
Sep 13	E-Commerce	Digital	OpenCost-Mock	Database	5.0583	USD
Sep 13	CRM	Sales	OpenCost-Mock	Database	4.5526	USD
Sep 13	AI Platform	AI	OpenCost-Mock	Object Storage	3.5425	USD
Sep 13	Data Platform	Data	OpenCost-Mock	Object Storage	3.2083	USD
Sep 13	E-Commerce	Digital	OpenCost-Mock	Object Storage	2.9189	USD
Sep 13	Data Platform	Data	OpenCost-Mock	Networking	2.1439	USD
Sep 13	AI Platform	AI	OpenCost-Mock	Networking	2.0137	USD
Sep 13	E-Commerce	Digital	OpenCost-Mock	Networking	1.8949	USD
Sep 13	CRM	Sales	OpenCost-Mock	Object Storage	1.8019	USD
Sep 13	CRM	Sales	OpenCost-Mock	Networking	1.1221	USD
Sep 12	Data Platform	Data	OpenCost-Mock	Compute	15.0332	USD
Sep 12	AI Platform	AI	OpenCost-Mock	Compute	11.3222	USD
Sep 12	Data Platform	Data	OpenCost-Mock	Database	8.9309	USD
Sep 12	E-Commerce	Digital	OpenCost-Mock	Compute	8.7126	USD

شکل ۴-۶. نمایش ۴۸۰ رکورد هزینه در Odoo پس از همگام‌سازی اولیه

4.4 راه‌اندازی Kubernetes و Helm

در Docker Desktop یک کلاستر Kubernetes از نوع kind با یک Node و نسخه 1.36.1 ساخته شد. دستور kubectl get nodes وضعیت Ready را برای plane-control-desktop نشان داد. انتخاب kind برای محیط توسعه محلی، سبک و مناسب بود.



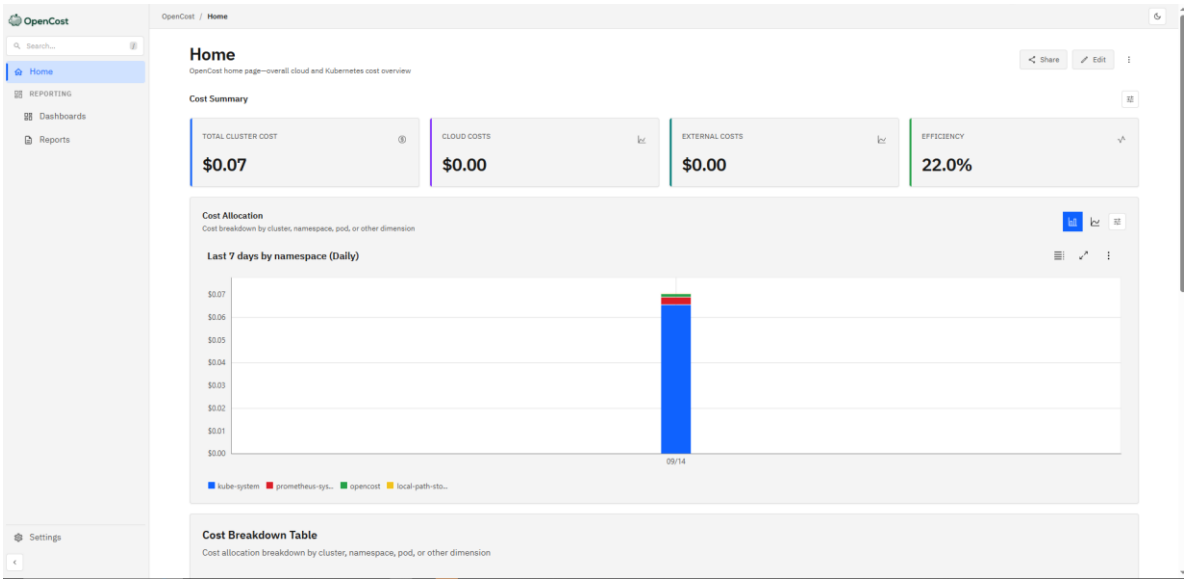
شکل ۴-۷. فعال بودن کلاستر Kubernetes از نوع kind در Docker Desktop

در مرحله بعد Helm نصب شد. به دلیل نبود winget در سیستم، فایل اجرایی رسمی Helm به صورت دستی در C:\helm قرار گرفت و مسیر آن به PATH اضافه شد. Prometheus ابتدا از GitHub Releases با خطای 504 روبه‌رو شد؛ سپس همان Chart از OCI روی ghcr.io نصب شد و وضعیت Release به deployed تغییر کرد.

4.5 نصب Prometheus و OpenCost

Prometheus در namespace جداگانه نصب و Podهای اصلی به وضعیت Running رسیدند. سپس OpenCost با Helm نصب شد. با Port Forward، رابط کاربری OpenCost روی پورت 9090 و API روی پورت 9003 در دسترس قرار گرفت.

```
kubectl port-forward --namespace opencost service/opencost 9003 9090
# UI:  http://localhost:9090
# API: http://localhost:9003/allocation?window=30m&aggregate=namespace&resolution=1m
```



شکل ۴-۸. رابط کاربری OpenCost و نمایش Cost Summary

4.6 ایجاد Workloadهای پروژه و تخصیص منابع

چهار namespace متناظر با پروژه‌های سازمانی ساخته شد و در هرکدام یک Deployment مبتنی بر nginx اجرا گردید. برای اینکه اختلاف تخصیص هزینه در OpenCost قابل مشاهده باشد، Resource Request و Limit متفاوت برای پروژه‌ها تعریف شد.

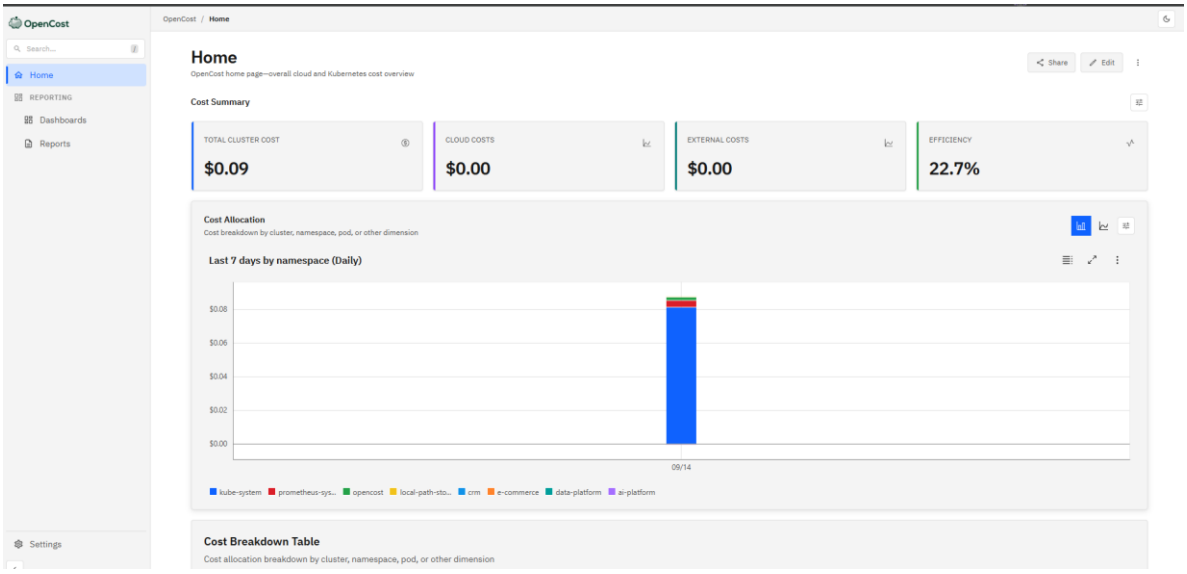
جدول ۴-۱. منابع درخواستی Workloadهای Kubernetes

پروژه	Namespace	CPU Request	Memory Request
CRM	crm	100m	64Mi
Commerce-E	commerce-e	200m	128Mi
Data Platform	platform-data	350m	256Mi
AI Platform	platform-ai	600m	512Mi

```
prometheus-system    prometheus-server-5749f59465-j5nxg              2/2     Running            0          148m
(.venv) PS C:\Users\ASUS\Desktop\uni project\finops-odoo-hub> kubectl get pods -A
NAMESPACE            NAME                                            READY   STATUS    RESTARTS   AGE
ai-platform          ai-app-7bd86556bf-4cbkf                         1/1     Running   0          4m30s
crm                  crm-app-5554b48555-wtbp5                        1/1     Running   0          4m42s
data-platform        data-app-7cdcc65d75-6kfm5                       1/1     Running   0          4m36s
e-commerce           web-5d6b5c7df9-gfwjn                            1/1     Running   0          4m48s
kube-system          coredns-589f44dc88-p2m8n                        1/1     Running   0          3h11m
kube-system          coredns-589f44dc88-zlqqp                        1/1     Running   0          3h11m
kube-system          etcd-desktop-control-plane                      1/1     Running   0          3h11m
kube-system          kindnet-rpkfn                                   1/1     Running   0          3h11m
kube-system          kube-apiserver-desktop-control-plane            1/1     Running   0          3h11m
kube-system          kube-controller-manager-desktop-control-plane   1/1     Running   0          3h11m
kube-system          kube-proxy-jvwlj                                1/1     Running   0          3h11m
kube-system          kube-scheduler-desktop-control-plane            1/1     Running   0          3h11m
local-path-storage   local-path-provisioner-855c7b7774-lzgxq         1/1     Running   0          3h11m
opencost             opencost-85dc5987dd-vgrvs                       2/2     Running   0          112m
prometheus-system    prometheus-kube-state-metrics-86f6b846bf-vp4h5  1/1     Running   0          153m
prometheus-system    prometheus-prometheus-node-exporter-sgtzf       1/1     Running   0          153m
prometheus-system    prometheus-server-5749f59465-j5nxg              2/2     Running   0          153m
(.venv) PS C:\Users\ASUS\Desktop\uni project\finops-odoo-hub>
```

شکل ۴-۹. Podهای پروژه و سرویس‌های OpenCost/Prometheus در وضعیت Running

پس از چند دقیقه جمع‌آوری متریک، OpenCost چهار namespace پروژه را در Cost Allocation تشخیص داد. هزینه‌های محیط محلی کوچک هستند، اما مشاهده پروژه‌ها در UI و API تایید می‌کند که جریان داده Kubernetes → Prometheus → OpenCost برقرار است.



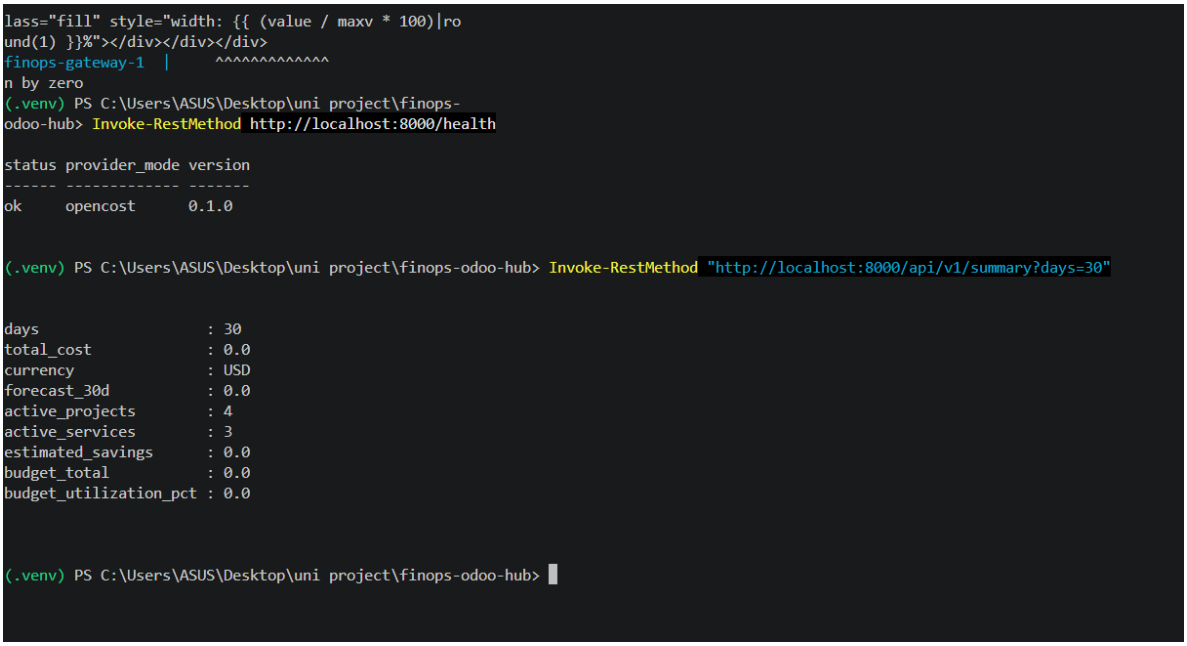
شکل ۴-۱۰. نمایش namespaceهای پروژه در Cost Allocation رابط OpenCost

4.7 تغییر Gateway از Mock به OpenCost

در ریشه پروژه فایل .env ساخته شد و متغیر MODE_PROVIDER_FINOPS از mock به opencost تغییر کرد. همچنین آدرس OpenCost برای Gateway داخل Docker برابر

health/ endpoint سرویس، recreate-force پس از 9003:internal.docker.host تنظیم شد. پس از مقدار opencost=mode_provider را برگرداند.

```
FINOPS_PROVIDER_MODE=opencost
FINOPS_OPENCOST_BASE_URL=http://host.docker.internal:9003
FINOPS_OPENCOST_WINDOW=1h
```



شکل ۴-۱۱. تایید mode_provider برابر opencost در endpoint سلامت Gateway

4.8 اصلاحات نرم‌افزاری برای Provider واقعی

Adapter اولیه OpenCost برای endpoint دیگری طراحی شده بود. برای سازگاری با محیط واقعی Kubernetes، منطق دریافت به /allocation تغییر یافت، aggregation بر اساس namespace انجام شد و هزینه‌های CPU، RAM، Storage و Network به مدل NormalizedCost نگاشت شدند. همچنین Seed داده ماکت فقط در mock=mode_provider اجرا می‌شود تا دیتابیس حالت واقعی با داده آزمایشی ترکیب نشود.

در Dashboard یک حالت مرزی نیز شناسایی شد. زمانی که دیتابیس تازه است و حداکثر مقدار نمودار برابر صفر می‌شود، عبارت maxv/value باعث ZeroDivisionError می‌شود. راهکار ایمن این است که در صورت صفر بودن maxv مقدار درصد نیز صفر در نظر گرفته شود.

```
{{ ((value / maxv * 100) if maxv else 0)|round(1) }}
```

5

فصل پنجم

آزمون، نتایج و ارزیابی سامانه

5 آزمون، نتایج و ارزیابی سامانه

آزمون پروژه در چند سطح انجام شده است: آزمون واحد و API در Gateway، آزمون اجرای چندسرویسی Docker، آزمون همگام‌سازی Odoo، آزمون سلامت Kubernetes و سرویس‌های مانیتورینگ، و آزمون Allocation API در OpenCost. نتایج این فصل وضعیت واقعی پروژه در زمان تهیه گزارش را نشان می‌دهد.

5.1 آزمون Gateway و API

در فاز Mock، تست‌های pytest پس از افزوده‌شدن پروژه AI Platform به‌روزرسانی شدند و خطای assert 4 == 3 با اصلاح انتظار تست رفع شد. Swagger امکان تست مستقل endpointها را فراهم کرد و endpoint /health برای کنترل Provider فعال استفاده شد. پس از تغییر تنظیمات، health مقدار opencost را گزارش کرد.

5.2 آزمون Odoo و Idempotency

اولین Sync ماکت به Odoo تعداد ۴۸۰ رکورد ایجاد کرد. اجرای مجدد Sync تعداد رکوردها را ثابت نگه داشت. این نتیجه نشان می‌دهد کلید id_external و منطق Upsert از Duplicate جلوگیری می‌کنند. همچنین داده‌ها در List View قابل مشاهده و در Graph/Pivot قابل تحلیل هستند.

5.3 آزمون Kubernetes و OpenCost

دستور kubectl get pods -A نشان داد Podهای چهار پروژه، kube-system، Prometheus و OpenCost همگی در حالت Running قرار دارند. Allocation API نیز با code=200 پاسخ داد و namespaceهای commerce-e، platform-data، crm و platform-ai را بازگرداند.

```
200
(.venv) PS C:\Users\ASUS\Desktop\uni project\finops-odoo-hub> $r.data[0].PSObject.Properties |
>> Where-Object { $_.Name -in @("e-commerce","data-platform","crm","ai-platform") } |
>> ForEach-Object {
>>     [PSCustomObject]@{
>>         Namespace = $_.Name
>>         TotalCost = [math]::Round($_.Value.totalCost, 6)
>>         CPUCost   = [math]::Round($_.Value.cpuCost, 6)
>>         RAMCost   = [math]::Round($_.Value.ramCost, 6)
>>     }
>> } | Format-Table

Namespace     TotalCost CPUCost RAMCost
---------     --------- ------- -------
ai-platform           0       0       0
crm                   0       0       0
data-platform         0       0       0
e-commerce      0.00001       0       0


(.venv) PS C:\Users\ASUS\Desktop\uni project\finops-odoo-hub>
```

شکل ۵-۱. خروجی Allocation API برای چهار namespace پروژه

مقادیر هزینه در محیط kind بسیار کوچک هستند و برخی مؤلفه‌ها پس از گردکردن تا شش رقم اعشار صفر دیده می‌شوند. این رفتار به معنای شکست محاسبه نیست؛ زیرا OpenCost در UI هزینه کل کلاستر را نمایش داده و namespaceهای پروژه را در Legend و API شناسایی کرده است.

5.4 نتیجه Gateway در حالت OpenCost و خطای مرزی Dashboard

پس از تنظیم .env و بازسازی Container، endpoint سلامت مقدار opencost را برگرداند. در دیتابیس تازه، Summary شامل cost_total=0 و total_budget=0 بود. صفحه Dashboard برای محاسبه عرض نوار از maxv/value استفاده می‌کرد و چون maxv برابر صفر بود، خطای 500 رخ داد. Traceback دقیقاً به عبارت Jinja2 در html.dashboard اشاره کرد.

```
finops-gateway-1  |     content = template.render(co
ntext)
finops-gateway-1  |   File "/usr/local/lib/python3.1
3/site-packages/jinja2/environment.py", line 1295, i
n render
finops-gateway-1  |     self.environment.handle_exce
ption()
finops-gateway-1  |     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~^^
finops-gateway-1  |   File "/usr/local/lib/python3.1
3/site-packages/jinja2/environment.py", line 942, in
 handle_exception
finops-gateway-1  |     raise rewrite_traceback_stac
k(source=source)
finops-gateway-1  |   File "/app/app/templates/dashb
oard.html", line 34, in top-level template code
finops-gateway-1  |     <div class="bar-row"><div cl
ass="bar-label"><span>{{ name }}</span><b>${{ '%.2f'
|format(value) }}</b></div><div class="track"><div c
lass="fill" style="width: {{ (value / maxv * 100)|ro
und(1) }}%"></div></div></div>
finops-gateway-1  |     ^^^^^^^^^^^^^
finops-gateway-1  | ZeroDivisionError: float divisio
n by zero
finops-gateway-1  | INFO:     172.18.0.1:35984 - "GE
T / HTTP/1.1" 500 Internal Server Error
finops-gateway-1  | ERROR:    Exception in ASGI appl
ication
finops-gateway-1  | Traceback (most recent call last
):
finops-gateway-1  |   File "/usr/local/lib/python3.1
3/site-packages/uvicorn/protocols/http/httptools_imp
l.py", line 421, in run_asgi
finops-gateway-1  |     result = await app(  # type:
 ignore[func-returns-value]
finops-gateway-1  |              ^^^^^^^^^^^^^^^^^^^
```

شکل ۵-۲. Traceback خطای ZeroDivisionError در html.dashboard هنگام maxv برابر صفر

رفع این خطا یک اصلاح مقاوم‌سازی UI است و ماهیت ارتباط OpenCost را تغییر نمی‌دهد. پس از اعمال شرط if maxv else 0، Dashboard باید در حالت بدون داده نیز به صورت پایدار رندر شود و سپس Sync واقعی OpenCost → Gateway → Odoo انجام می‌شود.

جدول ۵-۱. نتایج آزمون‌های اصلی

وضعیت	نتیجه	آزمون

موفق	APIها قابل دسترسی و پاسخ‌گو	Swagger / FastAPI
موفق	تست‌ها پس از تغییر پروژه اصلاح شدند	pytest در فاز Mock
موفق	PostgreSQL، Odoo و Gateway اجرا شدند	Docker Compose
موفق	در Odoo نصب و قابل استفاده	FinOps Connector
موفق	۴۸۰ رکورد؛ بدون Duplicate در Sync مجدد	Sync ماکت به Odoo
موفق	Node در وضعیت Ready	Kubernetes kind
موفق	Podهای اصلی Running	Prometheus
موفق	UI فعال؛ Allocation API با 200=code	OpenCost UI/API
موفق	چهار namespace در OpenCost دیده شدند	تشخیص پروژه‌ها
موفق	opencost = health/	Gateway provider
اصلاح نهایی	ZeroDivisionError شناسایی و راهکار مشخص شد	Dashboard با دیتابیس خالی
مرحله بعد	پس از اصلاح Dashboard و پاک‌سازی Mock	Sync واقعی Odoo⟶Gateway⟶OpenCost

5.5 تحلیل نتیجه

نتایج نشان می‌دهد معماری چندلایه انتخاب‌شده عملی است. هر لایه به طور مستقل قابل آزمون بوده و خطاها در محل صحیح شناسایی شده‌اند: مشکل WSL2 در لایه میزبان، timeout Prometheus در

مسیر دریافت Chart، و خطای تقسیم بر صفر در لایه نمایش. این جداسازی، عیب‌یابی و توسعه پروژه را ساده‌تر کرده است.

6

فصل ششم

جمع‌بندی و کارهای آینده

6 جمع‌بندی و کارهای آینده

پروژه از یک نمونه ساده مبتنی بر داده ماکت به یک معماری چندلایه متن‌باز توسعه یافته است. در وضعیت فعلی، Docker Compose، Odoo Community، FinOps Connector، Kubernetes kind، Prometheus و OpenCost همگی راه‌اندازی شده‌اند و Gateway به حالت opencost تغییر یافته است.

6.1 دستاوردهای اصلی

- پیاده‌سازی Gateway مستقل با FastAPI و REST API مستندشده.
- طراحی Provider Pattern برای جابه‌جایی بین Mock و OpenCost.
- راه‌اندازی Odoo Community و افزونه اختصاصی .FinOps Connector
- تایید Idempotency با ثابت ماندن ۴۸۰ رکورد در Sync مجدد.
- راه‌اندازی Kubernetes kind، Prometheus و OpenCost به صورت واقعی.
- تعریف چهار پروژه و Resource Requestهای متفاوت برای ایجاد سناریوی .Cost Allocation
- تایید اتصال Gateway به حالت opencost و ثبت عیب‌یابی‌های واقعی پروژه.

6.2 وضعیت تکمیل پروژه

جدول ۶-۱. وضعیت تکمیل اجزای پروژه

وضعیت	مولفه
تکمیل	Gateway Mock و Dashboard
تکمیل	API و Swagger
تکمیل	Docker Compose

تکمیل	PostgreSQL + Odoo
تکمیل	FinOps Connector
تکمیل	Sync ماکت و Idempotency
تکمیل	Kubernetes kind
تکمیل	Prometheus
تکمیل	OpenCost UI/API
تکمیل	چهار Workload پروژه
تکمیل	تغییر Gateway به opencost
راهکار مشخص؛ اعمال نهایی باقی‌مانده	رفع ZeroDivision Dashboard
مرحله بعد	Sync واقعی Gateway→OpenCost
مرحله بعد	پاک‌سازی Mock در Odoo و Sync واقعی
توسعه پیشنهادی	Budget مستقل از Mock
توسعه پیشنهادی	Scheduler و Deploy سازمانی

6.3 مراحل نهایی و توسعه‌های آینده

- اعمال شرط ایمن در html.dashboard و Rebuild کردن Gateway.
- اجرای POST /provider/sync/v1/api و تایید received > 0 برای OpenCost واقعی.

- بررسی /costs/v1/api و تایید provider=OpenCost و source=allocation-opencost.
- پاک‌سازی رکوردهای Mock-OpenCost در Odoo و اجرای Sync Now برای ورود داده واقعی.
- جداسازی Budget از Seed ماکت و تعریف آن به عنوان داده مدیریتی مستقل.
- افزودن Scheduled Sync، Authentication، لاگ ساختاریافته و مدیریت Secrets.
- تبدیل Mapping پروژه‌ها از دیکشنری ثابت به تنظیمات مدیریتی قابل ویرایش.
- توسعه Recommendation بر اساس نسبت usage/request، Anomaly Detection و Forecast.

6.4 نتیجه‌گیری

دستاورد اصلی پروژه، ایجاد یک مسیر متن‌باز و قابل توسعه برای FinOps است که از لایه زیرساخت Kubernetes تا لایه ERP ادامه پیدا می‌کند. برخلاف یک داشبورد صرف، این معماری شامل جمع‌آوری متریک، محاسبه Cost Allocation، نرمال‌سازی داده، API، ذخیره‌سازی، همگام‌سازی و تحلیل در Odoo است. نتیجه نشان می‌دهد می‌توان با ترکیب ابزارهای متن‌باز و یک لایه Gateway سبک، داده فنی هزینه را به اطلاعات قابل استفاده در فرایندهای مدیریتی تبدیل کرد.

منابع و مراجع

]1 [FinOps Foundation ;FinOps Framework ;https//:www.finops.org/framework/

]2 [OpenCost ;OpenCost Documentation ;https//:opencost.io/docs/

]3 [Cloud Native Computing Foundation ;Kubernetes Documentation ;
https//:kubernetes.io/docs/

]4 [Prometheus Authors ;Prometheus Documentation ;https//:prometheus.io/docs/

]5 [Odoo S.A. ;Odoo Documentation and Developer Reference ;
https//:www.odoo.com/documentation/

]6 [Sebastián Ramírez ;FastAPI Documentation ;https//:fastapi.tiangolo.com/

]7 [SQLAlchemy Authors ;SQLAlchemy Documentation ;https//:docs.sqlalchemy.org/

]8 [Docker Inc. ;Docker Documentation ;https//:docs.docker.com/

]9 [Helm Authors ;Helm Documentation ;https//:helm.sh/docs/

[10] مستند آموزشی «عملیات مالی FinOps – سامانه کاتب» ارائه‌شده به عنوان مبنای مفهومی اولیه پروژه.

[11] سورس کد و مستندات پروژه FinOps OpenCost Odoo Hub تهیه‌شده در این پروژه.

پیوست‌ها

پیوست الف: راهنمای اجرای پروژه و دستورات اصلی

این پیوست فرمان‌های اصلی را به ترتیب اجرایی جمع‌آوری می‌کند تا محیط پروژه از حالت خاموش تا وضعیت قابل ارائه راه‌اندازی شود.

الف-۱. اجرای Gateway محلی

```
cd finops-odoo-hub\gateway
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m pytest -q
uvicorn app.main:app --reload
```

الف-۲. اجرای Docker Compose

```
cd finops-odoo-hub
docker compose up -d --build
docker compose ps
# Gateway: http://localhost:8000
# Odoo:    http://localhost:8069
```

الف-۳. کنترل Kubernetes

```
kubectl get nodes
kubectl get pods -A
helm version
helm list -A
```

الف-۴. Port Forward و تست OpenCost

```
kubectl port-forward --address=0.0.0.0 -n opencost service/opencost 9003:9003 9090:9090
# UI:  http://localhost:9090
# API: http://localhost:9003/allocation?window=30m&aggregate=namespace&resolution=1m
$r = Invoke-RestMethod
'http://localhost:9003/allocation?window=30m&aggregate=namespace&resolution=1m'
```

```
$r.code
```

الف-۵. تنظیم Provider واقعی

```
Copy-Item .\.env.example .\.env
# .env
FINOPS_PROVIDER_MODE=opencost
FINOPS_OPENCOST_BASE_URL=http://host.docker.internal:9003
FINOPS_OPENCOST_WINDOW=1h


docker compose up -d --build --force-recreate finops-gateway
docker compose exec finops-gateway printenv FINOPS_PROVIDER_MODE
Invoke-RestMethod http://localhost:8000/health
```

الف-۶. عملیات نهایی Sync

```
Invoke-RestMethod -Method Post http://localhost:8000/api/v1/sync/provider
Invoke-RestMethod "http://localhost:8000/api/v1/costs?days=7"
# سپس در Odoo:
# FinOps -> Sync Now
```

پیوست ب: ماتریس خطاها و راه‌حل‌ها

جدول پ-۱. ماتریس خطاهای مهم و راه‌حل‌ها

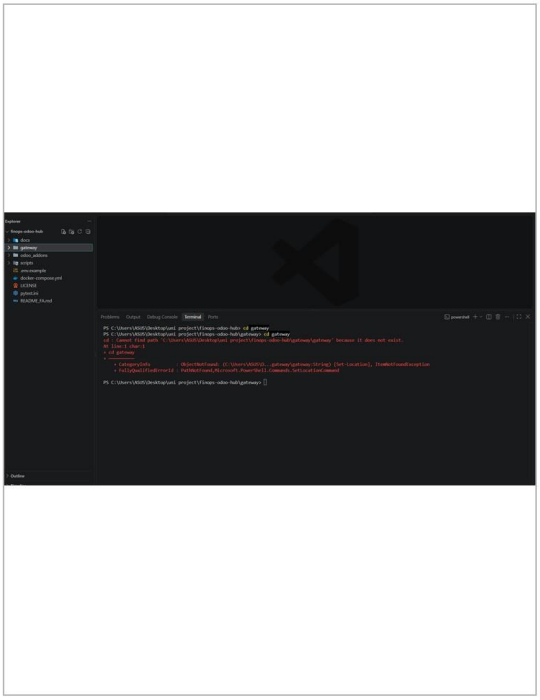
خطا	علت	راه‌حل
cd gateway دوباره	ترمینال از قبل داخل gateway بود	Location–Get و عدم اجرای cd مجدد
pytest is not recognized	pytest در venv نصب نبود	m pip install pytest– python
3 == assert 4	افزودن AI Platform و قدیمی بودن تست	به‌روزرسانی انتظار تست از 3 به 4

بررسی Task Manager، فعال‌سازی WSL2 و update	مشکل Kernel/WSL2 نه BIOS	Docker Virtualization not detected
download-web-- update-- wsl	Kernel نصب/به‌روز نبود	WSL2 kernel file not found
بازکردن Docker Desktop و انتظار برای Engine running	Docker Engine خاموش بود	Docker API pipe not found
نصب باینری رسمی و افزودن C:\helm به PATH	Helm نصب/در PATH نبود	helm not recognized
نصب دستی Helm	App Installer/winget موجود نبود	winget not recognized
استفاده از OCI روی io.ghcr	Timeout در GitHub Releases	Prometheus 504
استفاده از A- helm list	تفاوت Helm 4	all unknown-- helm list
env. example.env. Item-Copy	فایل تنظیمات واقعی ساخته نشده بود	env not found.
recreate Gateway-force و بررسی printenv	Container متغیر قدیمی داشت	mock هنوز provider
شرط if maxv else 0 در Jinja2	تقسیم بر صفر در 0=maxv	Dashboard 500

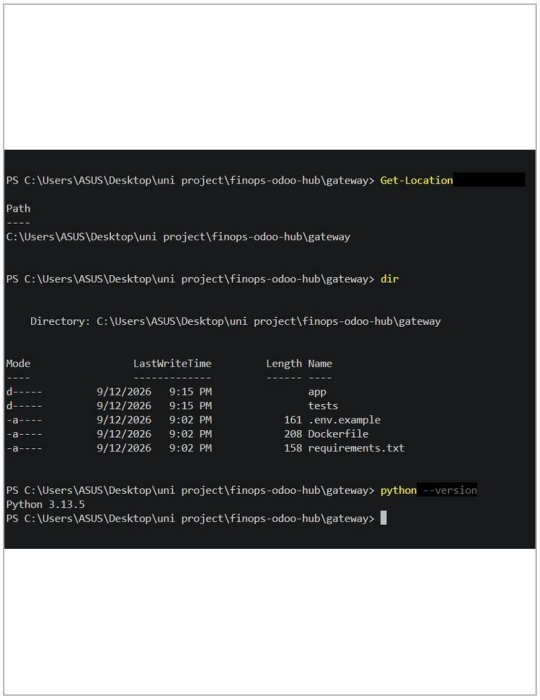
پیوست ج: مستند تصویری تکمیلی مراحل اجرا

برای حفظ خوانایی متن اصلی، تصاویر کلیدی در فصل‌های چهارم و پنجم با اندازه مناسب درج شده‌اند. در ادامه، تمام اسکرین‌شات‌های ثبت‌شده در طول پیاده‌سازی به صورت برگه‌های تماس ارائه می‌شوند. در تصویر ساخت دیتابیس Odoo، Master Password به صورت عمدی حذف شده است.

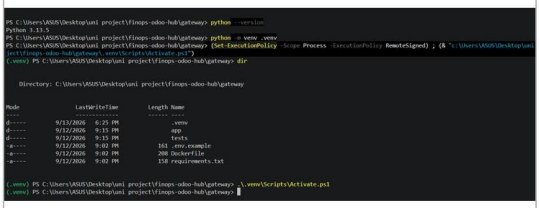
2026-09-13 182229



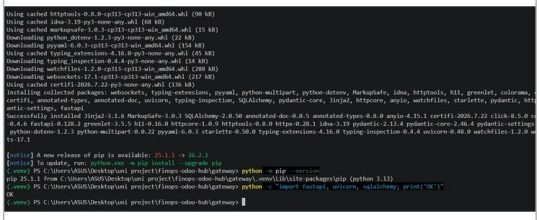
2026-09-13 182438



2026-09-13 182637

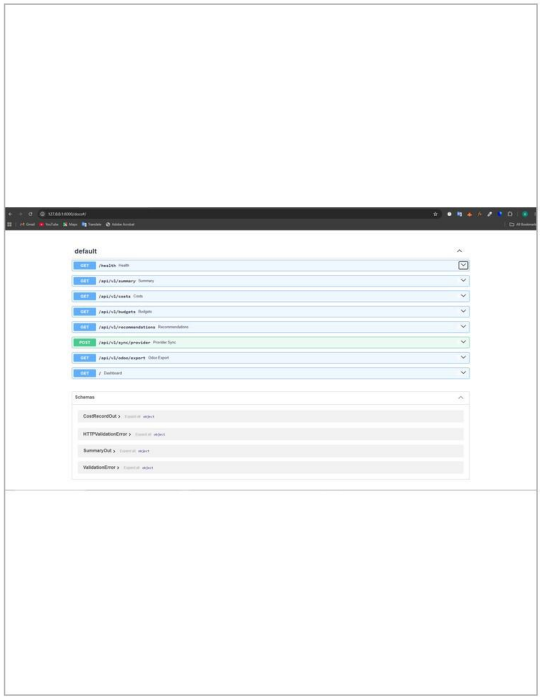


2026-09-13 183035

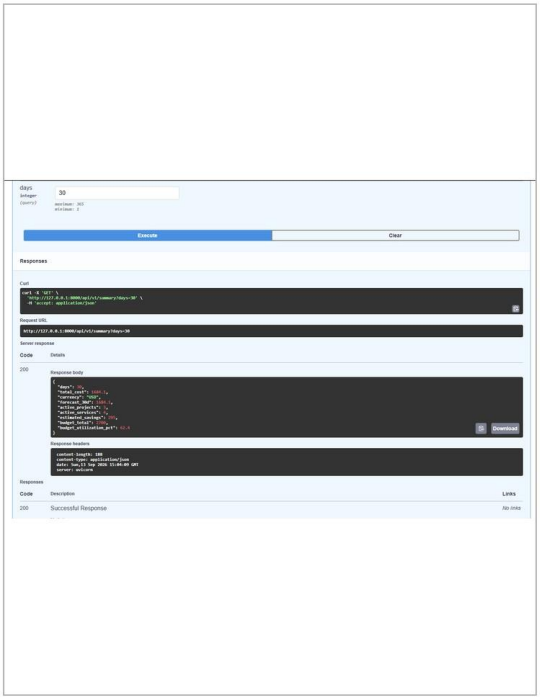


شکل پ-1. مجموعه تصاویر تکمیلی اجرای پروژه - بخش 1

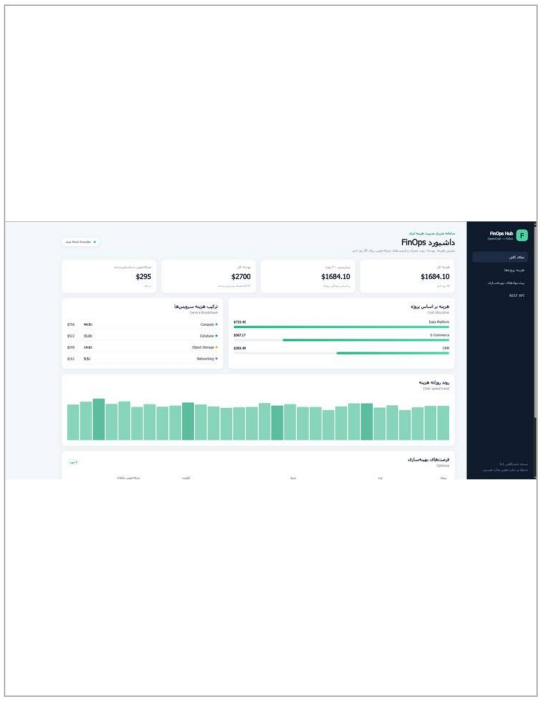
2026-09-13 183207



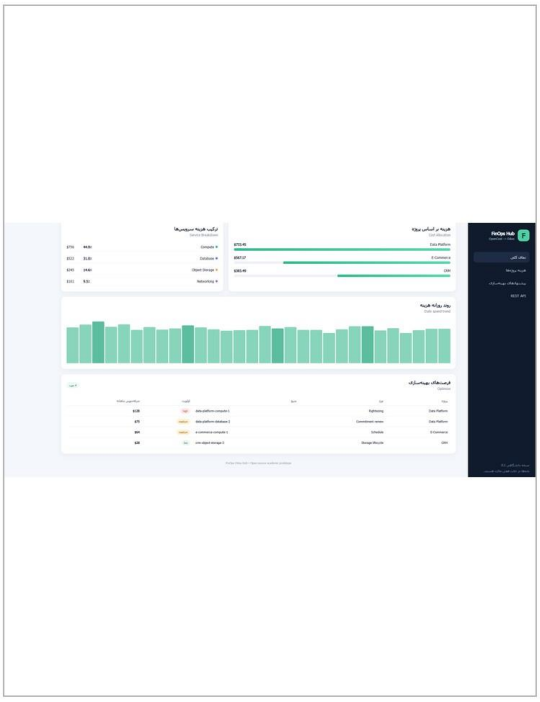
2026-09-13 183428



2026-09-13 183725

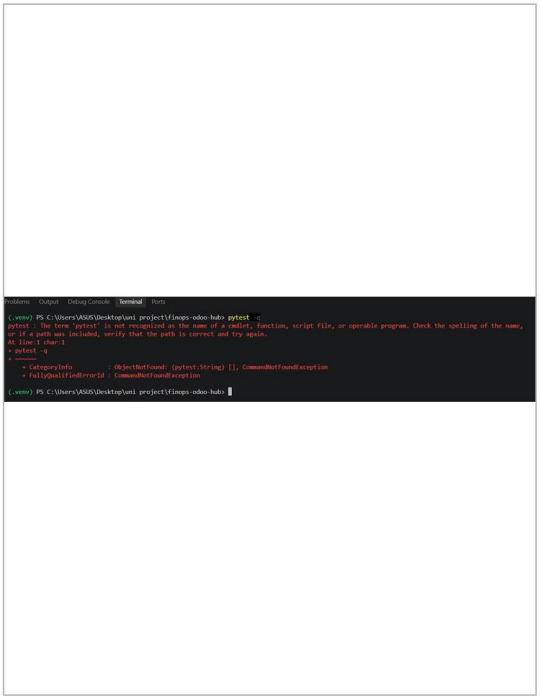


2026-09-13 183735

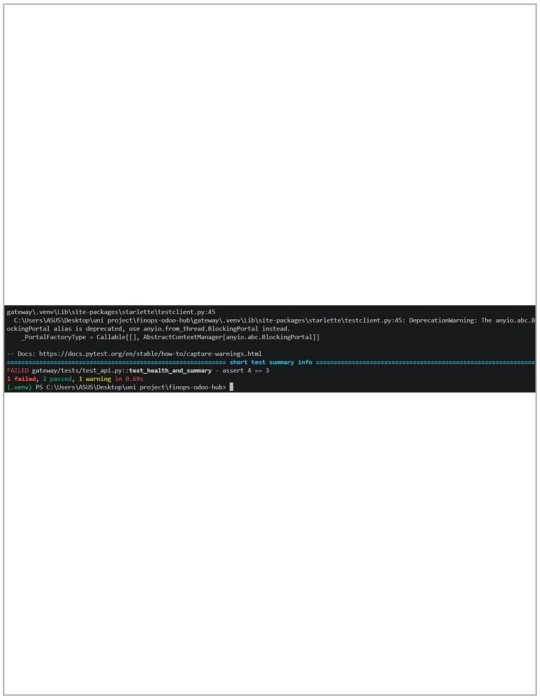


شکل پ-2. مجموعه تصاویر تکمیلی اجرای پروژه - بخش 2

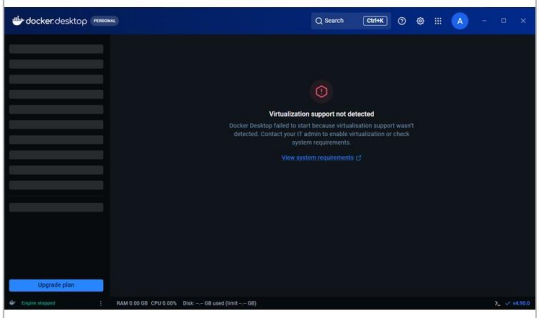
2026-09-13 202608



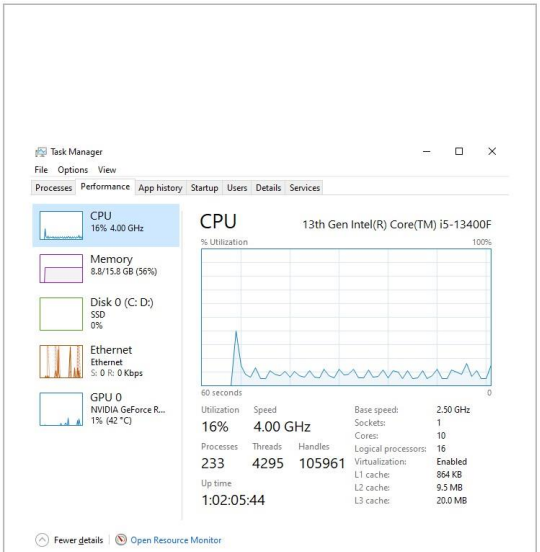
2026-09-13 202858



2026-09-13 204056

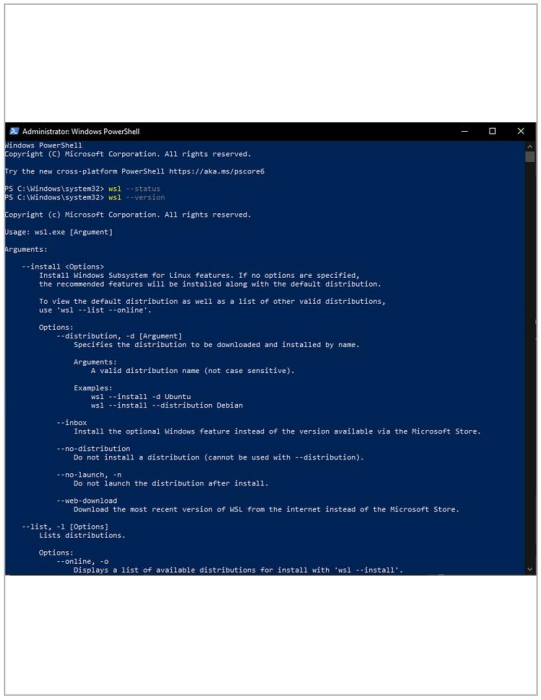


2026-09-13 204243

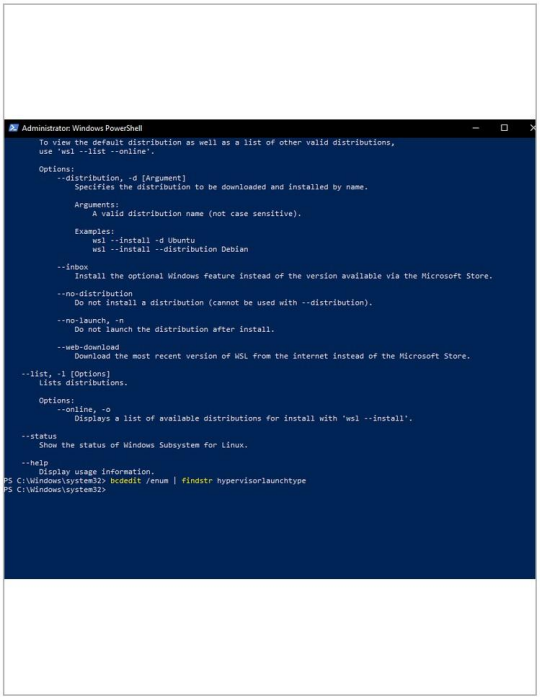


شکل پ-3. مجموعه تصاویر تکمیلی اجرای پروژه - بخش 3

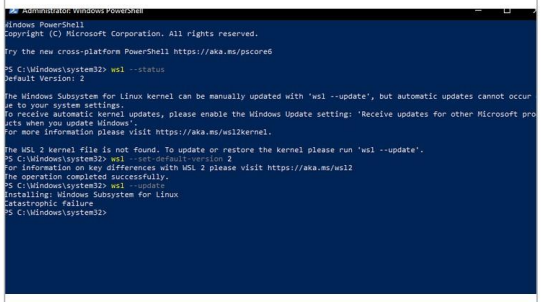
2026-09-13 204628



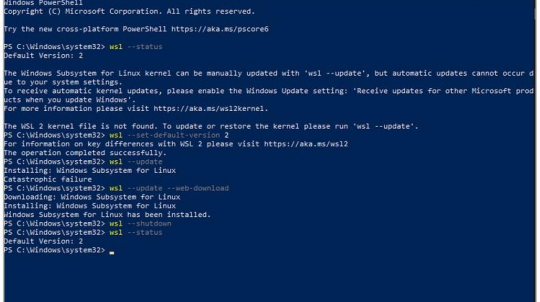
2026-09-13 204639



2026-09-13 205332

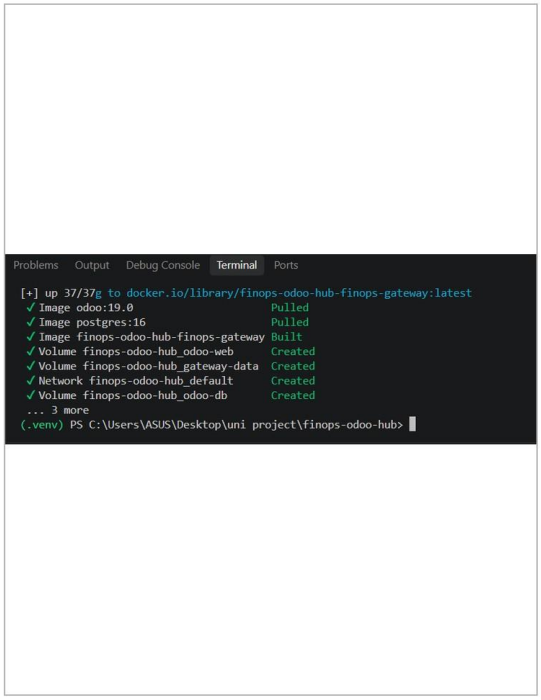


2026-09-13 215909

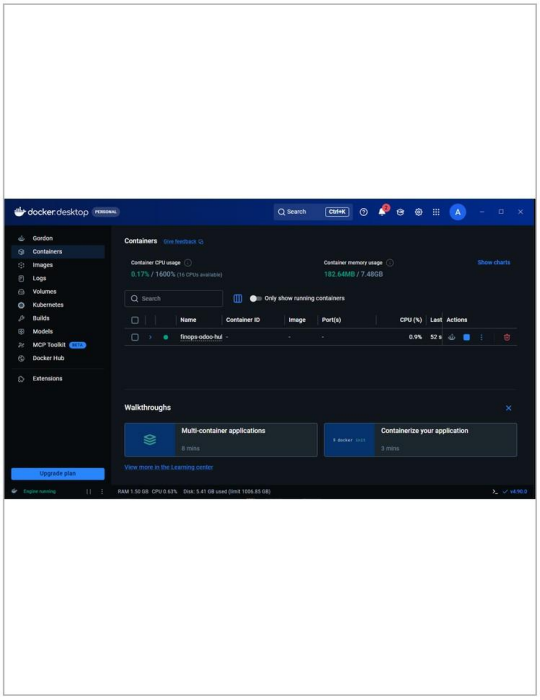


شکل پ-4. مجموعه تصاویر تکمیلی اجرای پروژه - بخش 4

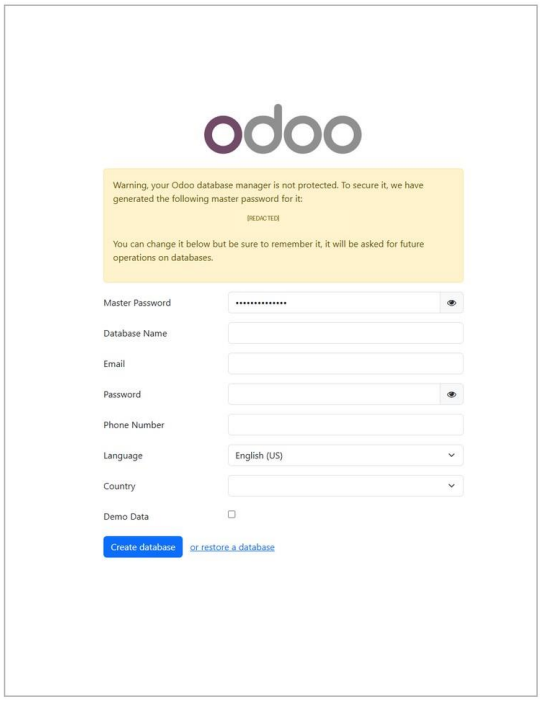
2026-09-13 233350



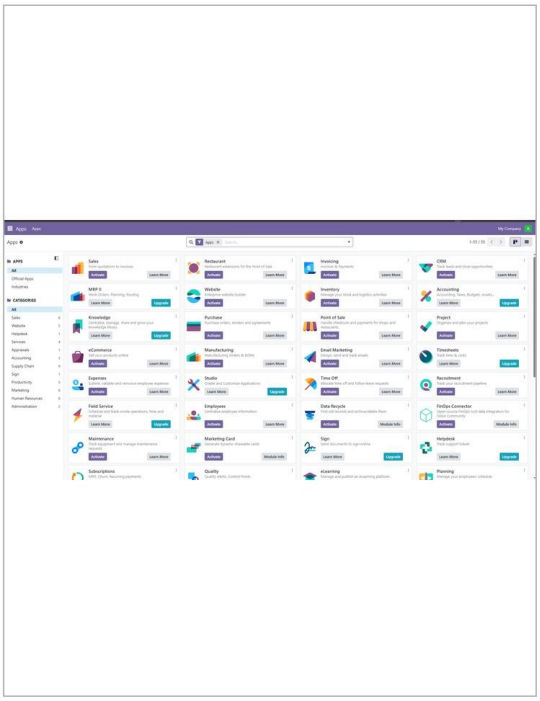
2026-09-13 233409



2026-09-13 233850

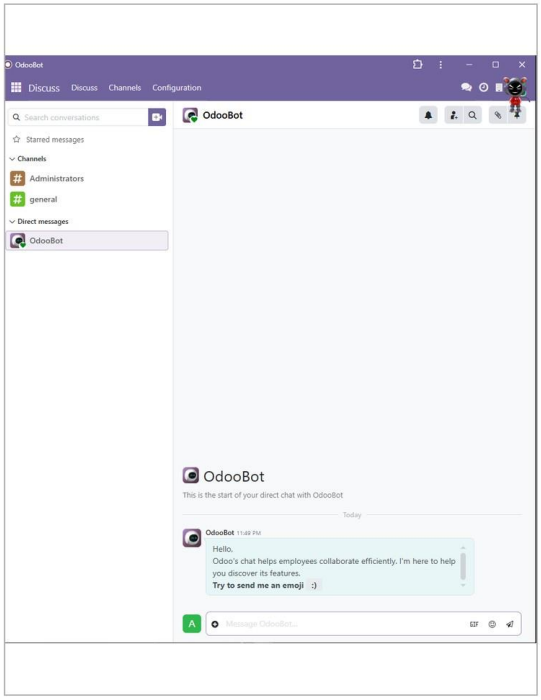


2026-09-13 234321

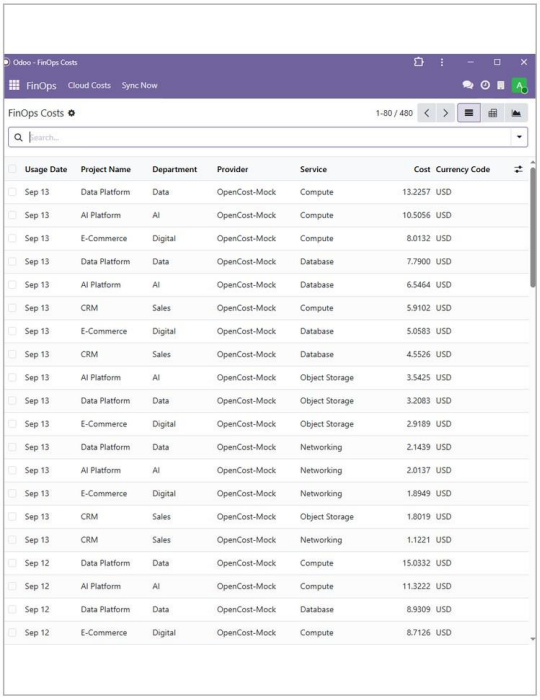


شکل پ-5. مجموعه تصاویر تکمیلی اجرای پروژه - بخش 5

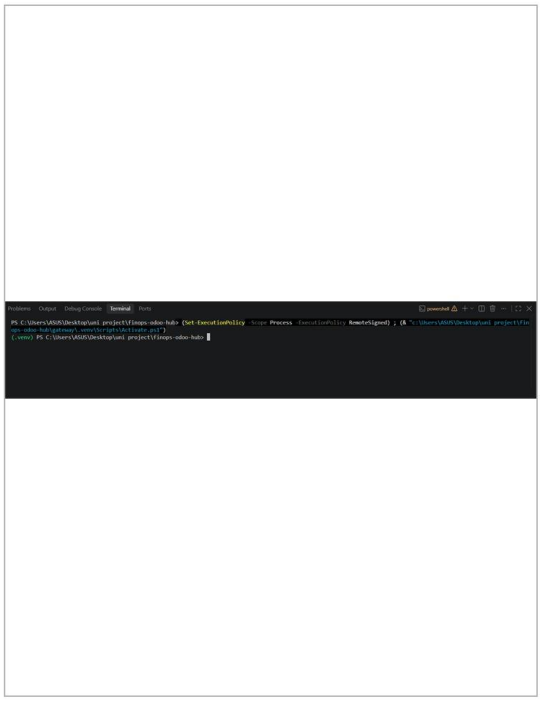
2026-09-13 235031



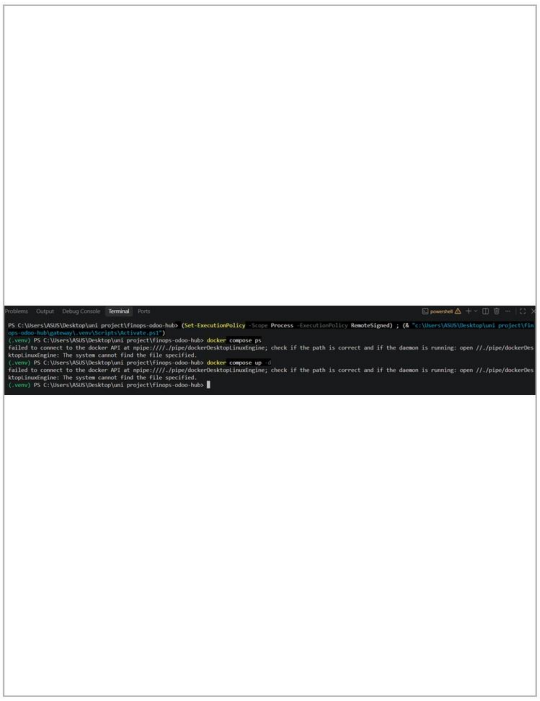
2026-09-13 235229



2026-09-14 164746

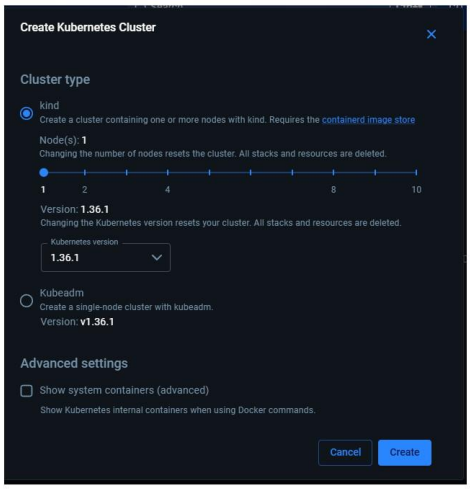


2026-09-14 165442



شکل پ-6. مجموعه تصاویر تکمیلی اجرای پروژه - بخش 6

2026-09-14 171754



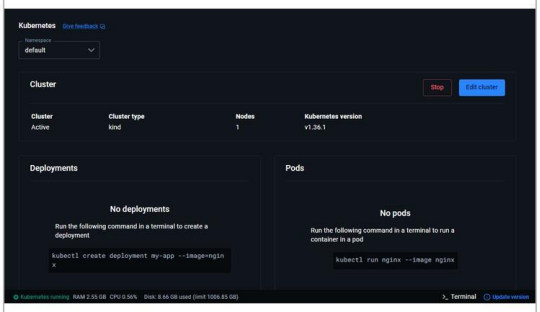
2026-09-14 173904



2026-09-14 173914

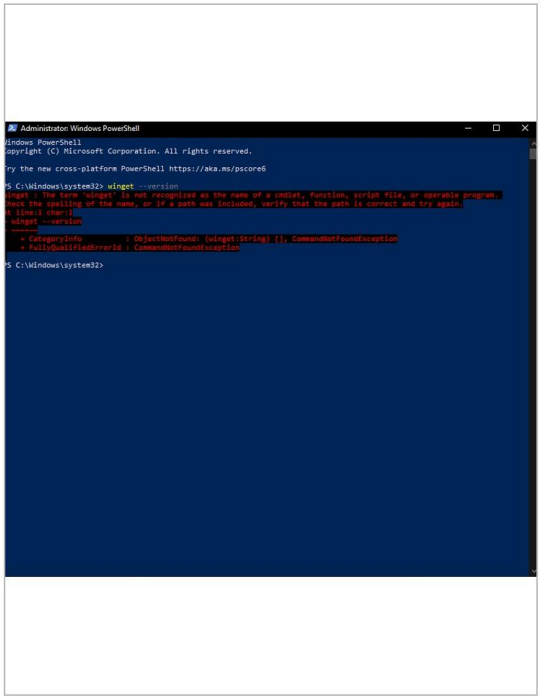


2026-09-14 173931

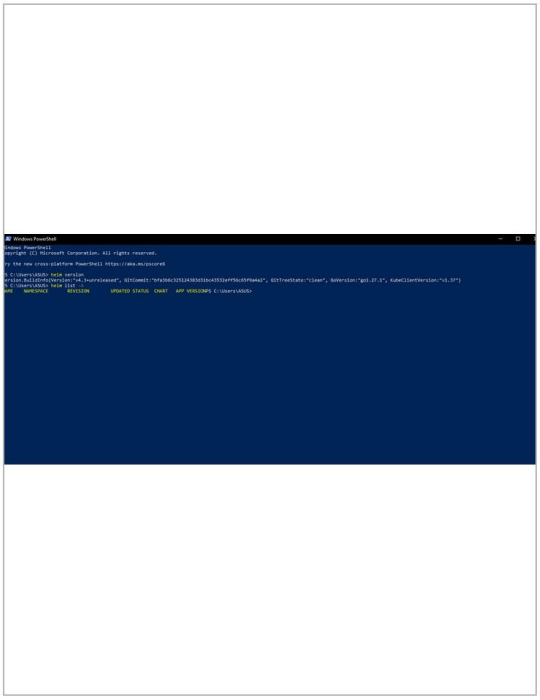


شکل پ-7. مجموعه تصاویر تکمیلی اجرای پروژه - بخش 7

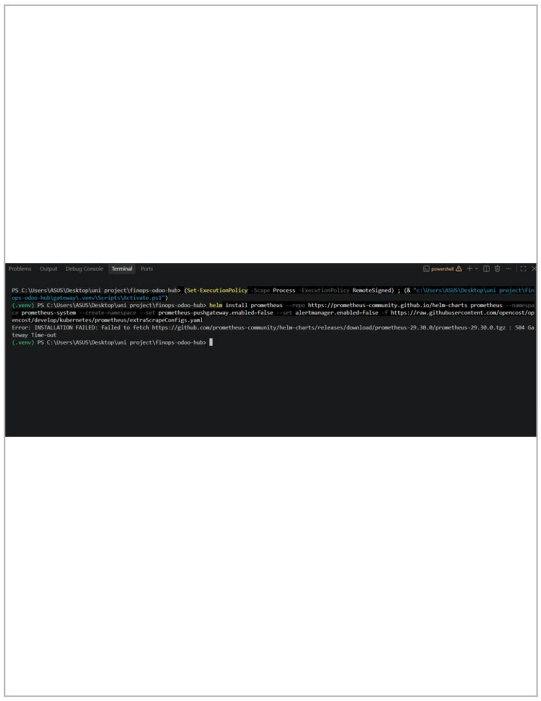
2026-09-14 174721



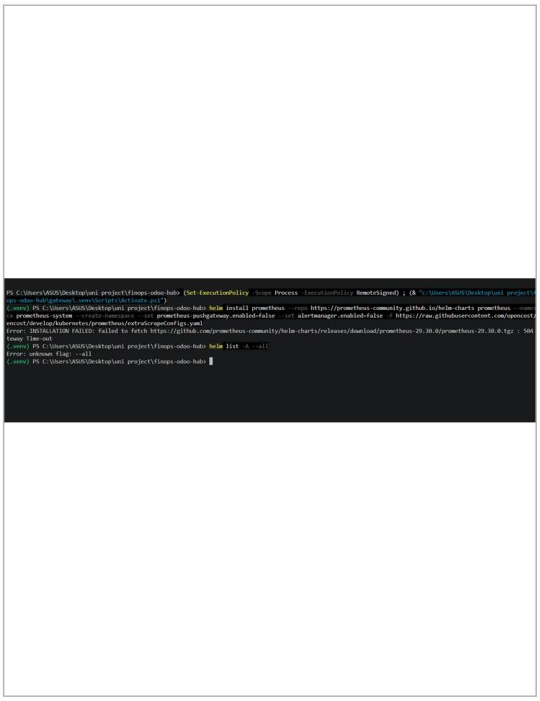
2026-09-14 180154



2026-09-14 181106



2026-09-14 181241

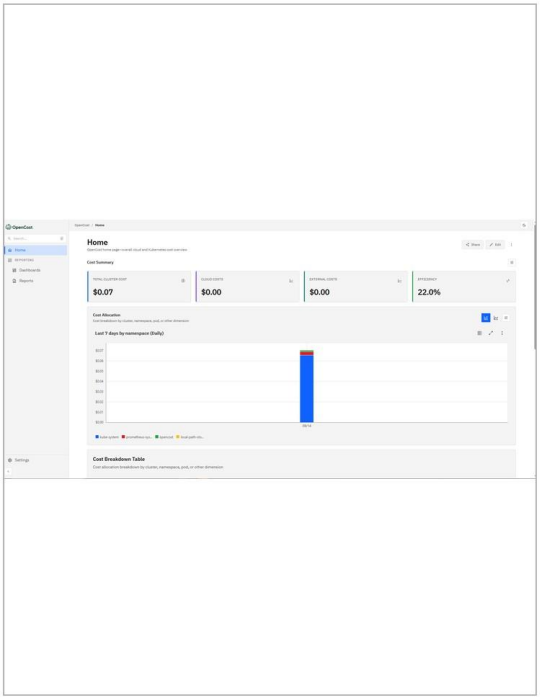


شکل پ-8. مجموعه تصاویر تکمیلی اجرای پروژه - بخش 8

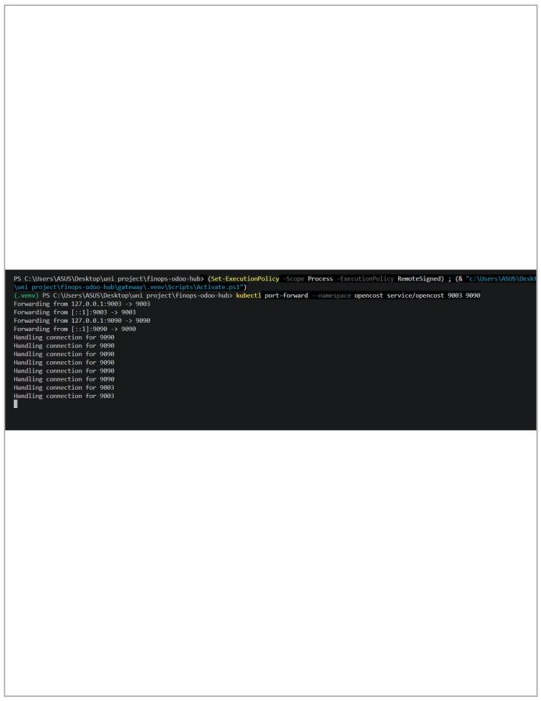
2026-09-14 181608



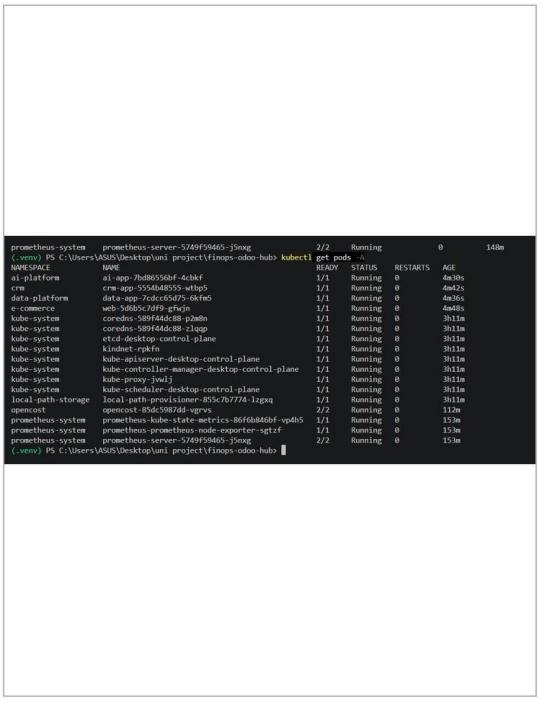
2026-09-14 202655



2026-09-14 202749

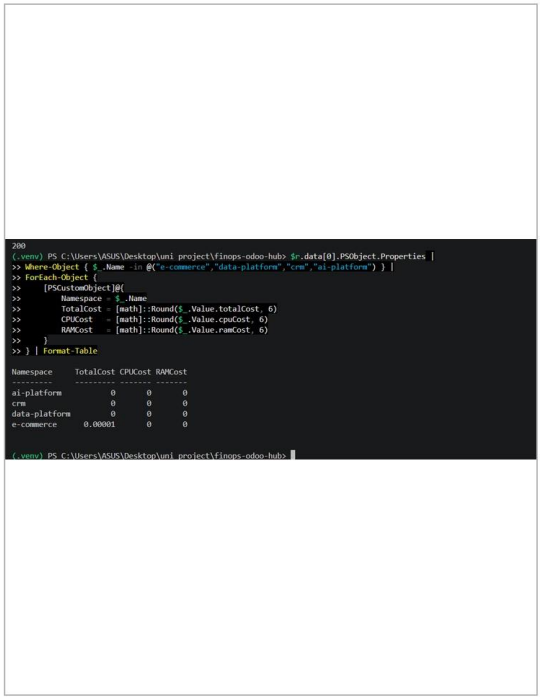


2026-09-14 204825

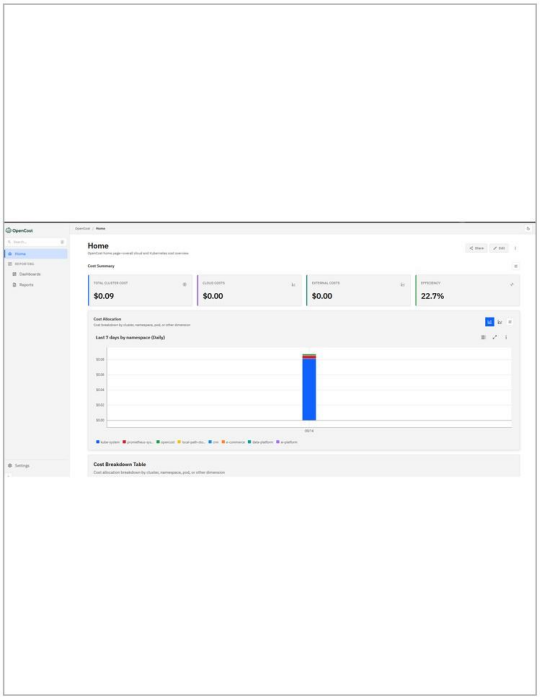


شکل پ-9. مجموعه تصاویر تکمیلی اجرای پروژه - بخش 9

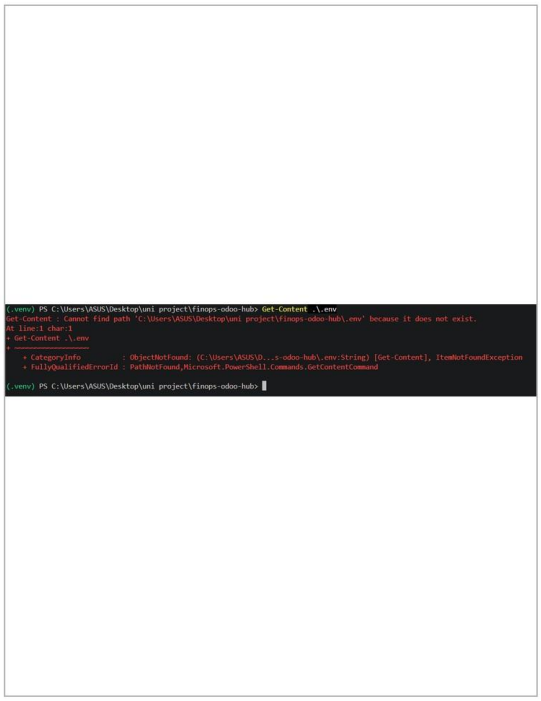
2026-09-14 205541



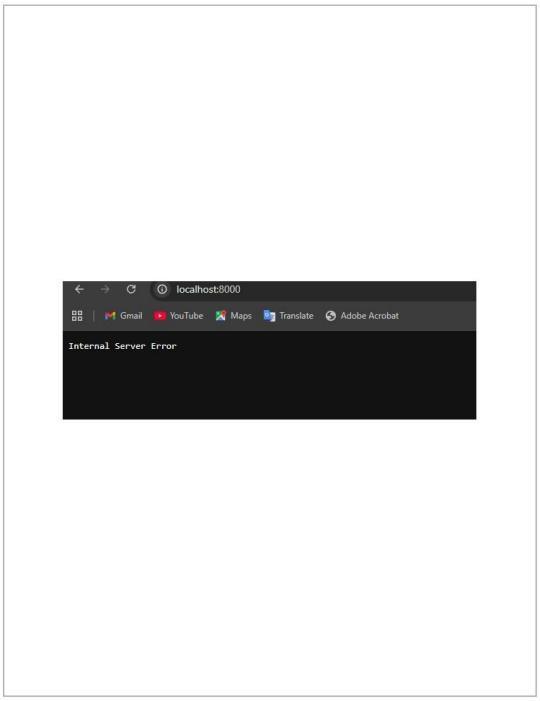
2026-09-14 205552



2026-09-14 212251

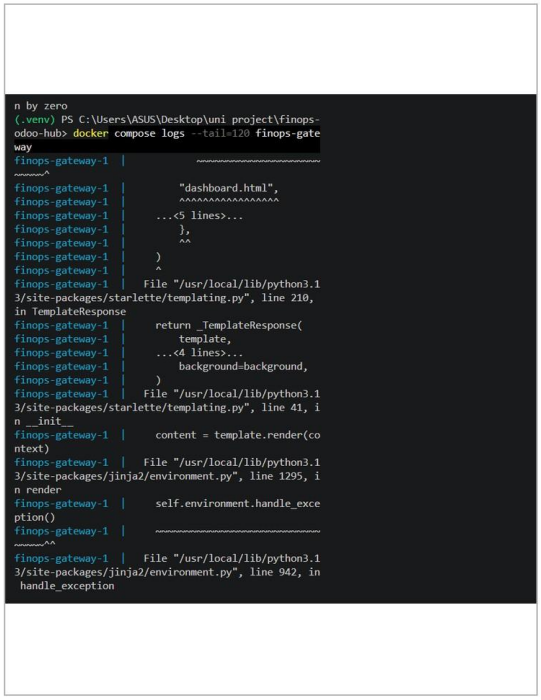


2026-09-14 215601

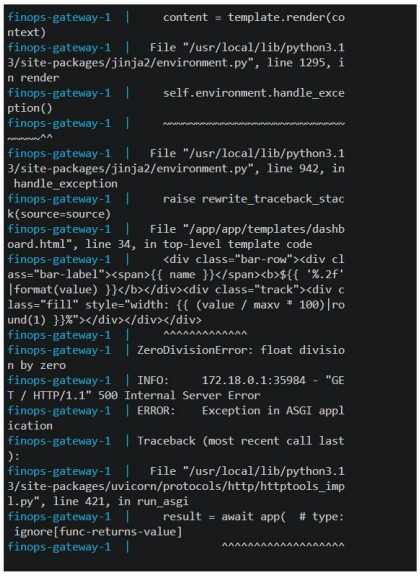


شکل پ-10. مجموعه تصاویر تکمیلی اجرای پروژه - بخش 10

2026-09-14 221346



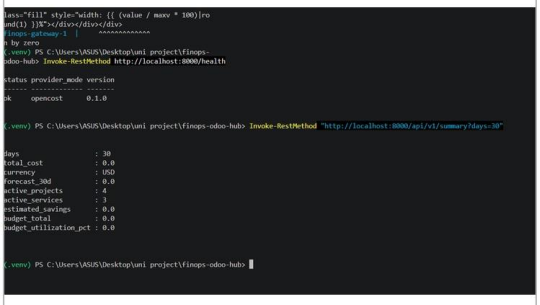
2026-09-14 221408



2026-09-14 221439



2026-09-14 221448



شکل پ-11. مجموعه تصاویر تکمیلی اجرای پروژه - بخش 11